



TRAKYA ÜNİVERSİTESİ
MÜHENDİSLİK FAKÜLTESİ
BİLGİSAYAR MÜHENDİSLİĞİ BÖLÜMÜ

BLM4502 YAPAY SİNİR AĞLARINA GİRİŞ
PROJE RAPORU

PROJE ADI:

Akıllı Park

PROJE EKİBİ:

1221602028 – İrem Su ERDEMİR

1221602043- Hatice Pınar YILMAZ

1221602098- Beyza YILMAZ

1221602641 – Yusuf GÜDE

1221602804 – Ulaş Ekrem EMİLİ

PROJE DANIŞMANI:

Dr. Öğr. Üyesi Turgut DOĞAN

Edirne -2026

İÇİNDEKİLER

1. GİRİŞ & PROJENİN TANITIMI.....	1
1.1. Akıllı Şehirlerde Otopark Problemi	1
1.2. Projenin Amacı ve Kapsamı	2
1.3. Parking Birmingham Veri Setinin Seçilme Gerekçesi.....	2
1.4. Projenin Özgün Yönleri.....	3
1.5. Hibrit Derin Öğrenme ve Pekiştirmeli Öğrenme Mimarisi.....	3
1.6. Gerçek Hayattaki Kullanım Senaryoları	4
1.7. Trafik Yoğunluğu ve Park Arama Davranışı.....	5
1.8. LSTM ve PPO Entegrasyonunun Teknik Gerekçesi.....	6
2. GEREKSİNİM ANALİZİ	7
2.1. Platform ve İşletim Sistemi Gereksinimleri	7
2.2. Kaynak Kısıtları ve Çözüm Önerileri	7
2.3. Çözülen Problemler ve Avantajlar	8
2.4. Hedef Kullanıcı Kitlesi.....	9
2.5. Kullanılan Teknolojiler ve Gerekçeleri.....	9
2.6. Destek Alınan Ancak Projede Yer Almayan Teknolojiler	10
3. TASARIM.....	11
3.1. Genel Sistem Mimarisi.....	11
3.2. Veri Ön İşleme ve Özellik Mühendisliği	12
3.3. LSTM Pipeline.....	13
3.4. RL Pipeline ve PPO Eğitim Mantığı.....	13
3.5. Durum Uzayı (State Space)	14
3.6. Eylem Uzayı (Action Space)	14
3.7. Ödül Fonksiyonu ve Reward Shaping.....	14

3.8.	Grid Environment ve Dynamic Occupancy	15
3.9.	Salınım Problemi ve Stabilizasyon	15
3.10.	Explainable AI ve Streamlit Dashboard.....	15
3.11.	Veri Akış Diyagramı ve Sequence Generation.....	16
3.12.	Static Goal vs Dynamic Goal Tartışması	16
3.13.	Action Masking ve Geçersiz Eylem Yönetimi	17
3.14.	DQN Eğitim Yapılandırması.....	17
3.15.	SWOT Analizi	18
4.	GELİŞTİRME	19
4.1.	Veri Hazırlama ve Özellik Mühendisliği Uygulaması.....	19
4.2.	LSTM Tabanlı Doluluk Tahmin Modelinin Gerçekleştirilmesi	24
4.3.	RL Ortamının Gerçekleştirilmesi.....	26
4.4.	Ödül Sistemi ve PPO Eğitim Süreci	29
4.5.	Dashboard ve Görselleştirme Araçlarının Geliştirilmesi	32
4.5.1.	Streamlit Tabanlı Kullanıcı Arayüzü	33
4.6.	Kurulum ve Çalıştırma Yönergeleri:	41
5.	TEST ve DOĞRULAMA	42
5.1.	LSTM Tahmin Performansı.....	42
5.2.	RL Ajan Performansı	43
5.3.	PPO vs DQN Analizi	46
5.4.	Baseline Karşılaştırması	47
5.5.	Oscillation ve Reward Shaping Etkisi.....	49
5.6.	Ajan Animasyonları ve Görsel Doğrulama	49
5.7.	Deney Tekrarlanabilirliği	52
6.	SONUÇ	53
6.1.	Gelecek Çalışmalar (Future Work).....	54

KAYNAKLAR	55
-----------------	----

ŞEKİLLER LİSTESİ

Şekil 1 Parking Birmingham Veri Setinde Kronolojik Doluluk Oranı Zaman Serisi Grafiği	3
Şekil 2 Akıllı Park Hibrit Sistem Mimarisi ve Veri Akış Diyagramı	4
Şekil 3 Saatlik Ortalama Otopark Doluluk Oranı Grafiği	5
Şekil 4 Doluluk Oranı Dağılım Histogram Grafiği	6
Şekil 5 . Zaman Bazlı Train/Validation/Test Bölme Stratejisi (%70/%15/%15)	12
Şekil 6 Otopark Lotları × Saat Dilimi Doluluk Isı Haritası.....	12
Şekil 7 Gün × Saat Doluluk Isı Haritası (RL senaryo analizi için)	15
Şekil 8 Özellikler Arası Korelasyon Matrisi.....	16
Şekil 9 Otopark Doluluk Verisi Doğrulama ve Temizleme	19
Şekil 10 Zaman Serisine Uygun Train/Validation/Test Bölme	20
Şekil 11 Özellik Mühendisliği ve Ölçeklenecek Değişkenlerin Tanımlanması	21
Şekil 12 Zaman Damgasına Göre Sızıntısız Veri Etiketleme	22
Şekil 13 Uçtan Uca İşlenmiş Veri Seti Oluşturma Pipeline'ı	23
Şekil 14 Çok Katmanlı LSTM ile Doluluk Tahmin Modeli	24
Şekil 15 LSTM Eğitim Döngüsü, Doğrulama Takibi ve Erken Durdurma	25
Şekil 16 Grid-Nav Bölüm Konfigürasyonlarının Zaman Diliminden Üretilmesi.....	27
Şekil 17 RL Gözlem Vektörünün Çok Kanallı Oluşturulması	28
Şekil 18 RL Ortamında step() ile Eylem Uygulama ve Ceza Yönetimi.....	29
Şekil 19 Grid Ortamı İçin Bileşenli Ödül Hesaplama Fonksiyonu	30
Şekil 20 PPO Ajanı İçin Eğitim Hiperparametrelerinin Yapılandırılması	30
Şekil 21 RL Modelinin Eğitilip Kaydedilmesi ve Normalizasyon Parametrelerinin Dondurulması.....	31
Şekil 22 RL Eğitim Süresi ve Ödül- Ceza Katsayılarının Kalibrasyonu.....	31
Şekil 23 Streamlit Arayüzünde Modüler Sekme Yapısının Oluşturulması	32
Şekil 24 RL Ödül Değerlerinin Açıklanması.....	33

Şekil 25 Akıllı Park Streamlit Arayüzü Genel Görünümü	33
Şekil 26 Zaman Serisi Tahmin ve Model Performansı Ekranı	34
Şekil 27 Gerçek Doluluk Oranı ile Tahmin Sonuçlarının Karşılaştırılması	35
Şekil 28 Keşifsel Veri Analizi Sekmesi.....	35
Şekil 29 Sistem Durumu ve Özet Göstergeler Dashboard Ekranı	36
Şekil 30 PPO/DQN Eğitim Grafiği Arayüzü	37
Şekil 31 RL Simülasyonunda Aksiyon Seçimi ve Grid Görselleştirmesi	38
Şekil 32 Ajan Karar Günlüğü Tablosu	39
Şekil 33 Adım Bazlı Ödül Değişim Grafiği.....	39
Şekil 34 Karar Destek ve Açıklanabilir Otopark Önerisi Ekranı	40
Şekil 35 LSTM Modeli Gerçek ve Tahmin Edilen Doluluk Oranı Karşılaştırması.....	42
Şekil 36 PPO Ajanının Hedef Otoparka Ulaşma Performansı.....	44
Şekil 37 PPO ve DQN Algoritmalarının Çoklu Metrik Karşılaştırması	44
Şekil 38 PPO ve DQN Eğitim Sürecinde Episode Reward Trend Eğrileri	45
Şekil 39 . Eğitim Sürecinde Başarı Oranı Proxy Trendi (ödül > 0)	45
Şekil 40 Algoritmaların Ortalama Rota Uzunluğu Karşılaştırması	45
Şekil 41 PPO ve DQN Karşılaştırmasında DQN Ajan Davranışı.....	46
Şekil 42 Greedy Shortest Path Baseline Sonucu.....	47
Şekil 43 Random Baseline Sonucu	48
Şekil 44 Hafta İçi ve Hafta Sonu Doluluk Karşılaştırması (EDA doğrulama)	49
Şekil 45 PPO Eğitim Süreci Başlangıç Davranışı.....	50
Şekil 46 PPO Eğitim Süreci Orta Aşama	50
Şekil 47 PPO Eğitim Süreci Sonunda Kararlı Politika.....	51
Şekil 48 En Yoğun 10 Otopark Lotu Analizi.....	51
Şekil 49 En Boş 10 Otopark Lotu Analizi	52

1. GİRİŞ & PROJENİN TANITIMI

1.1. Akıllı Şehirlerde Otopark Problemi

Hızla büyüyen kentlerde araç sahipliğinin artması, sürücülerin park yeri ararken harcadıkları süreyi ve buna bağlı olarak trafik yoğunluğunu önemli ölçüde artırmaktadır. Literatürde yapılan çalışmalar, şehir merkezlerindeki trafik yoğunluğunun önemli bir kısmının uygun park alanı arayan araçlardan kaynaklandığını göstermektedir [2]. Bu durum yalnızca zaman kaybına değil; yakıt tüketimi, karbon salınımı ve sürücü stresi gibi ekonomik ve çevresel problemlere de yol açmaktadır.

Akıllı şehir kavramı; sensör ağları, veri analitiği ve yapay zekâ teknolojilerini kullanarak kentsel hizmetlerin daha verimli hale getirilmesini amaçlamaktadır. Otopark yönetimi ise akıllı şehir uygulamalarının en önemli bileşenlerinden biri olarak değerlendirilmektedir[10]. Çünkü otopark doluluk bilgilerinin doğru zamanda analiz edilmesi ve sürücülere etkin biçimde sunulması, hem bireysel sürüş deneyimini hem de şehir içi trafik akışını doğrudan etkilemektedir.

Günümüzde birçok otopark sistemi yalnızca anlık doluluk bilgisi sunmaktadır. Ancak bu yaklaşım, sürücünün kısa vadede hangi bölgelerde yoğunluğun azalacağını öngörmesine yardımcı olmamaktadır. Benzer şekilde mevcut navigasyon sistemleri yoğunlukla statik hedef noktalar üzerinden yönlendirme yapmakta, geleceğe yönelik doluluk eğilimlerini dikkate almamaktadır.

Bu proje kapsamında, gerçek şehir otopark verileri kullanılarak kısa vadeli doluluk tahmini yapabilen ve bu tahminleri pekiştirmeli öğrenme tabanlı bir yönlendirme sistemiyle birleştiren hibrit bir yapay zekâ mimarisi geliştirilmiştir. Böylece sistem yalnızca mevcut duruma tepki veren değil, gelecekte oluşabilecek yoğunluğu da dikkate alarak proaktif yönlendirme yapabilen bir karar destek mekanizması sunmaktadır.

1.2. Projenin Amacı ve Kapsamı

Akıllı Park projesi, Trakya Üniversitesi Bilgisayar Mühendisliği Bölümü BLM4502 Yapay Sinir Ağlarına Giriş dersi kapsamında geliştirilmiştir. Projenin temel amacı, Parking Birmingham veri setinden elde edilen gerçek otopark doluluk kayıtlarını kullanarak kısa vadeli doluluk tahmini yapmak ve bu tahminleri pekiştirmeli öğrenme tabanlı bir yönlendirme ajanına entegre etmektir. Geliştirilen sistem, sürücüyü mevcut ve tahmin edilen doluluk koşullarını dikkate alarak en uygun, yani en düşük yoğunluklu hedef otoparka yönlendirmeyi amaçlamaktadır.

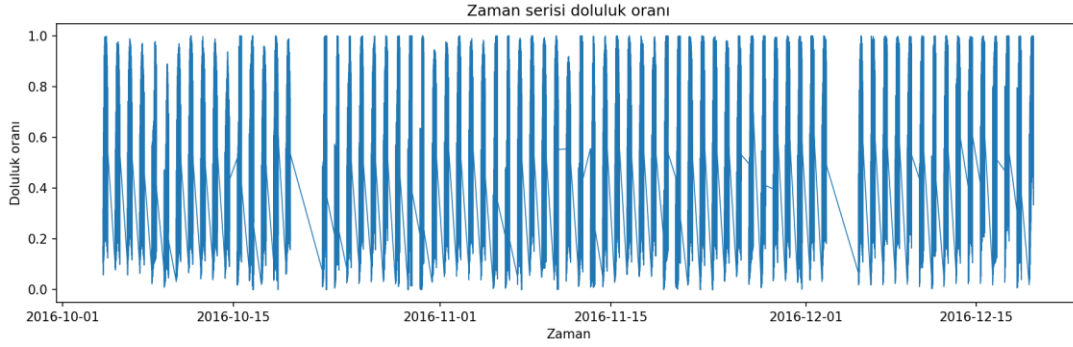
Proje kapsamında veri hazırlama, veri temizleme ve keşifsel veri analizi çalışmaları gerçekleştirilmiştir. Zaman serisi tabanlı doluluk tahmini için LSTM mimarisi kullanılmış, ayrıca karşılaştırmalı analiz amacıyla Random Forest ve XGBoost gibi tabular baseline modelleri de değerlendirilmiştir[7],[8]. Pekiştirmeli öğrenme tarafında ise Gymnasium tabanlı 15×15 ızgara ortamında PPO ve DQN algoritmaları eğitilmiş ve performans karşılaştırmaları yapılmıştır.

Bunun yanı sıra sistemin kullanıcıya görsel ve etkileşimli biçimde sunulabilmesi amacıyla Streamlit tabanlı bir karar destek paneli geliştirilmiş; FastAPI arka uç servisi ve React tabanlı ön uç bileşenleri ile istemci-sunucu mimarisine uygun bir prototip yapı oluşturulmuştur. Tüm deney çıktıları 'output/' ve 'evaluation/reports/' klasörlerinde saklanmış; bu raporda yalnızca gerçek metrikler ve üretilmiş görseller kullanılmıştır.

Bu yönüyle proje, yalnızca doluluk tahmini yapan bir makine öğrenmesi uygulaması olmanın ötesine geçerek; veri analizi, tahminleme, karar verme ve kullanıcı etkileşimi bileşenlerini bir araya getiren bütünsel bir akıllı park yönetim sistemi sunmaktadır.

1.3. Parking Birmingham Veri Setinin Seçilme Gerekçesi

Proje başlangıcında İzmir merkezli canlı veri toplama altyapısı değerlendirilmiş; ancak örneklem büyüklüğünün sınırlı olması, kesintisiz veri akışı için ek donanım gereksinimi ve öğrenci bütçesi kısıtları nedeniyle bu yaklaşımdan vazgeçilmiştir. Metodolojik tutarlılık, yeterli örneklem büyüklüğü ve deneylerin yeniden üretilebilirliği açısından Parking Birmingham veri seti tercih edilmiştir [1]. Veri seti; SystemCodeNumber (otopark kimliği), Capacity, Occupancy ve LastUpdated alanlarını içermekte; çoklu otopark lokasyonları üzerinden saatlik/periyojik doluluk kayıtları sunmaktadır. Bu yapı, LSTM tabanlı zaman serisi tahmini ve RL ortamında dinamik doluluk haritası oluşturma için uygundur.



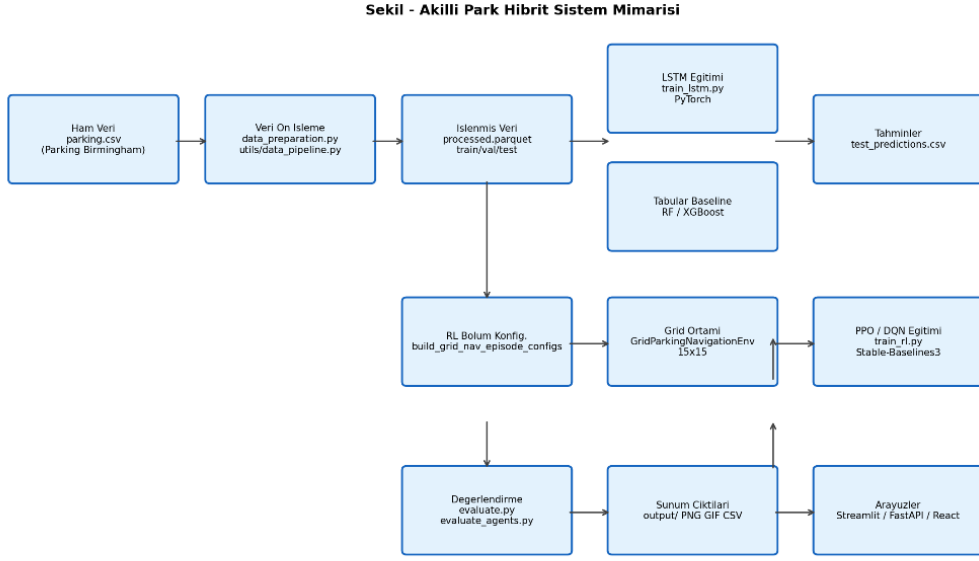
Şekil 1 Parking Birmingham Veri Setinde Kronolojik Doluluk Oranı Zaman Serisi Grafiği

1.4. Projenin Özgün Yönleri

Literatürdeki birçok çalışma yalnızca doluluk sınıflandırması veya tahminine odaklanmaktadır. Bu projenin özgün değeri, tahmin modeli çıktılarının PPO/DQN tabanlı RL ajanının gözlem vektörüne skaler kanal olarak bağlanmasıdır. Böylece sistem yalnızca mevcut doluluk haritasını değil, LSTM aggregate tahminini de dikkate alarak karar verir. Ayrıca maliyet duyarlı, bileşenleri ayrıştırılabilir (explainable) bir ödül fonksiyonu; Manhattan mesafe şekillendirme, salınım (oscillation) cezası, yüksek doluluk lotu geçiş cezası ve BFS optimaline yakınlık bonusu gibi mekanizmalar birlikte kullanılmıştır. Streamlit panelinde her RL adımının ödül bileşenleri kullanıcıya açıklanabilir biçimde sunulmaktadır.

1.5. Hibrit Derin Öğrenme ve Pekiştirmeli Öğrenme Mimarisi

Sistem mimarisi iki ana hattan oluşur. Birinci hat (denetimli öğrenme), `utils/data\_pipeline.py` ile üretilen lag, rolling istatistik ve zaman özelliklerini kullanarak 12 zaman adımlık sliding window ile bir sonraki aggregate doluluk oranını tahmin eder. PyTorch tabanlı çok katmanlı LSTM mimarisi (`utils/forecast\_models.py`) bu görevi üstlenir. İkinci hat (pekiştirmeli öğrenme), her benzersiz zaman damgası için bir RL bölümü oluşturur; lotlar hash tabanlı koordinatlarla 15×15 ızgara üzerine yerleştirilir ve hedef lot o anki en düşük doluluk oranına sahip alandır. LSTM tahmini `lstm\_aggregate\_pred` skaleri olarak gözleme eklenir; `predicted\_emptying\_norm` ise hedef lot doluluğunu temsil eder.



Şekil 2 Akıllı Park Hibrit Sistem Mimarisi ve Veri Akış Diyagramı

PPO (Proximal Policy Optimization) algoritması, Stable-Baselines3 kütüphanesi ile 280.000 zaman adımı boyunca eğitilmiştir. VecNormalize ile gözlem ve ödül normalizasyonu uygulanmış; Monitor ile episode logları `logs/monitor/ppo/` altına yazılmıştır. DQN ile karşılaştırmalı deneyler aynı ortam ve hiperparametre yapılandırması altında yürütülmüştür.

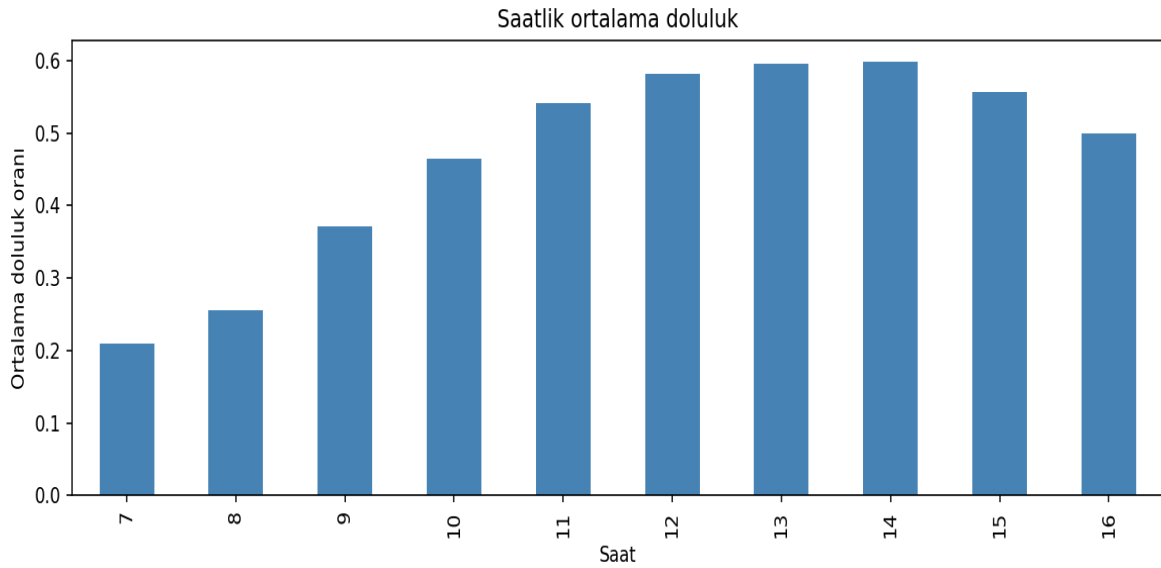
1.6. Gerçek Hayattaki Kullanım Senaryoları

- Alışveriş merkezi otoparklarında yoğun saatlerde sürücüyü alternatif giriş veya daha az dolu kata yönlendirme.
- Hastane kampüslerinde acil durum ziyaretçilerinin boş alana hızlı erişimi.
- Üniversite kampüslerinde ders saatlerine göre dinamik park önerisi.
- Belediye akıllı şehir platformlarına entegre edilebilecek API tabanlı karar destek servisi.
- Mobil uygulama veya navigasyon sistemlerine kısa vadeli doluluk tahmini beslemesi.

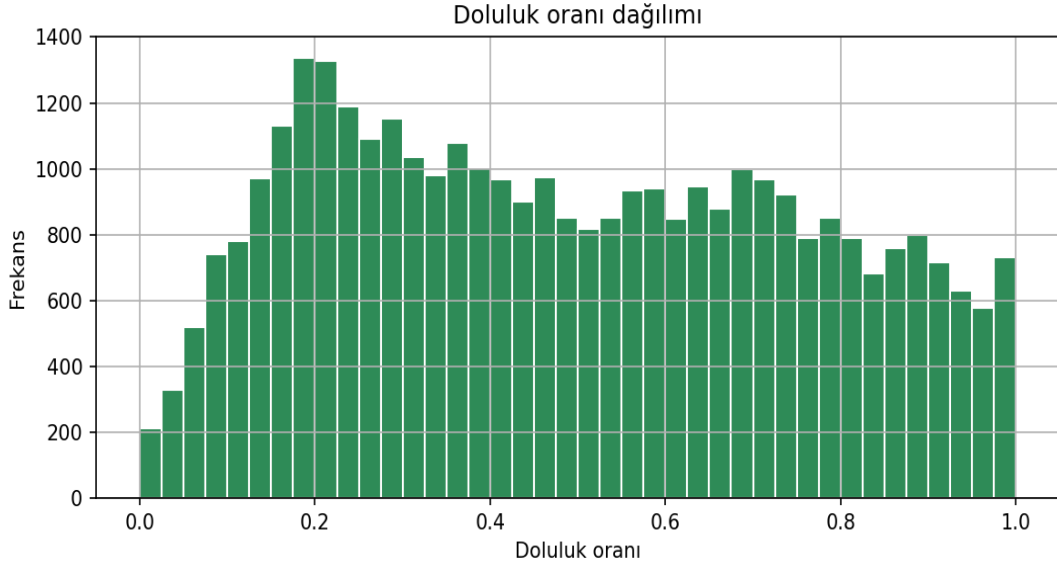
Geliştirilen FastAPI (`api/main.py`) ve React (`frontend/`) bileşenleri, Streamlit panelinin yanı sıra üretim ortamına yakın bir dağıtım senaryosunu destekleyecek şekilde tasarlanmıştır.

1.7. Trafik Yoğunluğu ve Park Arama Davranışı

Keşifsel veri analizi sonuçları, otopark doluluk oranlarının gün içinde belirgin periyodik desenler gösterdiğini ortaya koymuştur. Saatlik ortalama doluluk grafiği (`output/2_saatlik_ortalama.png`) sabah ve öğle saatlerinde pikler; gece saatlerinde düşüş eğilimi sergilemektedir. Hafta içi ve hafta sonu karşılaştırması ise kullanım alışkanlıklarının farklılaştığını göstermektedir. Bu bulgular, yalnızca anlık doluluk yerine zaman serisi tahmininin neden gerekli olduğunu destekler: sürücü hedefe varmadan önce doluluk değişebilir



Şekil 3 Saatlik Ortalama Otopark Doluluk Oranı Grafiği



Şekil 4 Doluluk Oranı Dağılım Histogram Grafiği

1.8. LSTM ve PPO Entegrasyonunun Teknik Gerekçesi

LSTM modelinin aggregate seri üzerinde eğitilmesi, lot bazlı gürültüyü yumuşatarak genel şehir doluluk eğilimini yakalar. RL ajanına aktarılan ``lstm_aggregate_pred`` skaleri, ajanın 'gelecekte genel doluluk artacak mı?' sorusuna kısa vadeli cevap almasını sağlar. Hedef lot seçimi ise o anki gerçek doluluk oranları üzerinden en düşük yoğunluklu alan belirlenerek yapılır; böylece tahmin ve anlık ölçüm birlikte kullanılır. Bu entegrasyon, künyede planlanan hibrit mimarinin kod tabanında (``env/grid_navigation_env.py``, satır 105–117 `GridNavEpisodeConfig`) somutlaştırılmıştır.

PPO algoritmasının DQN'e tercih edilmesi, yüksek boyutlu gözlem uzayında politika gradyan yöntemlerinin daha az instabilite göstermesi beklentisine dayanmaktadır. Deney sonuçları bu beklentiye doğrulamıştır: PPO %92,5 başarı oranına ulaşırken DQN %27,5 ile sınırlı kalmıştır. Entegrasyon sayesinde sistem yalnızca reaktif değil, proaktif karar verme kapasitesine kavuşmuştur.

2. GEREKSİNİM ANALİZİ

2.1. Platform ve İşletim Sistemi Gereksinimleri

Akıllı Park, masaüstü geliştirme ve sunum ortamında çalışacak şekilde tasarlanmış bir yazılım projesidir. Python tabanlı makine öğrenmesi bileşenleri Windows, Linux ve macOS işletim sistemlerinde sanal ortam içinde çalıştırılabilir. Web arayüzleri tarayıcı tabanlıdır; React geliştirme sunucusu ve Streamlit uygulaması yerel ağ veya localhost üzerinden erişilebilir. FastAPI arka uç servisi sunucu/ masaüstü platformlarında uvicorn ile barındırılabilir.

Gereksinim	Değer / Açıklama
Platform	Masaüstü (geliştirme), sunucu (API), tarayıcı (UI)
İşletim sistemi	Windows 10+, Ubuntu 20.04+, macOS 12+
Python	3.10+ (3.11 önerilir)
Node.js	18+ (React frontend için)
Bellek	Minimum 8 GB RAM (RL eğitimi için 16 GB önerilir)
Disk	~2 GB (veri + modeller + çıktılar)
GPU	Opsiyonel (PyTorch CPU ile çalışır)

TABLO 2.1. Platform ve Teknoloji Gereksinimleri

2.2. Kaynak Kısıtları ve Çözüm Önerileri

Proje geliştirme sürecinde karşılaşılan başlıca kısıtlar; veri erişimi, donanım kapasitesi, eğitim süresi ve model kararlılığı ile ilgili teknik problemler olmuştur. İlk olarak, canlı İzmir otopark verisine erişim ve kenar cihaz donanımı bütçesi önemli bir kısıt oluşturmuştur. Gerçek zamanlı veri toplama altyapısının maliyetli olması ve sürekli veri akışının sağlanamaması nedeniyle, çalışma kapsamında Parking Birmingham statik veri seti kullanılmıştır. Bu veri seti, düzenli zaman serisi yapısı ve yeterli örneklem büyüklüğü sayesinde deneylerin daha kontrollü ve tekrar üretilebilir biçimde gerçekleştirilmesine olanak sağlamıştır.

Pekiştirmeli öğrenme tarafında karşılaşılan bir diğer önemli problem RL eğitim süresidir. PPO ve DQN algoritmalarının yüksek timestep değerlerinde eğitilmesi, kullanılan donanımına bağlı olarak saatler sürebilmektedir. Özellikle 280.000 timestep hedefi bazı sistemlerde uzun eğitim sürelerine neden olmuştur. Bu problemi azaltmak amacıyla --timestepsparmetresi kullanılarak düşük timestep değerleriyle hızlı test ve deneme senaryoları oluşturulmuştur.

Bir diğer teknik kısıt ise yüksek boyutlu gözlem uzayıdır. RL ortamında kullanılan çok kanallı grid yapısı, doluluk ısı haritaları ve LSTM tahmin sinyalleri nedeniyle gözlem vektörü 1127 boyuta ulaşmıştır. Bu durum eğitim sırasında kararsızlığa neden olabildiğinden, VecNormalize yöntemi ile gözlem normalizasyonu uygulanmış ve ödül kırpma (GRID_REWARD_CLIP=550) mekanizması kullanılarak öğrenme süreci daha stabil hale getirilmiştir.

Ayrıca DQN algoritmasının bazı senaryolarda uzun episode timeout davranışı göstermesi ve hedefe ulaşmada kararsız hareketler sergilemesi önemli bir problem olarak gözlemlenmiştir. Bu nedenle PPO algoritması ile karşılaştırmalı analizler gerçekleştirilmiş ve reward shaping yöntemleri ile iyileştirme stratejileri uygulanmıştır. Özellikle Manhattan mesafe tabanlı yönlendirme, tekrar ziyaret cezası ve salınım (oscillation) cezaları sayesinde ajanın gereksiz dolaşma davranışı azaltılarak daha kararlı hareket etmesi sağlanmıştır.

2.3.Çözülen Problemler ve Avantajlar

Akıllı Park projesi; sürücülerin körlemesine park yeri arama süresini azaltma, trafik yoğunluğunu düşürme, kısa vadeli otopark doluluk tahmini yapma ve açıklanabilir pekiştirmeli öğrenme kararları sunma problemlerine çözüm üretmeyi amaçlamaktadır. Özellikle şehir merkezlerinde sürücülerin uygun park alanı bulabilmek için uzun süre dolaşması, hem trafik yoğunluğunu artırmakta hem de zaman, yakıt ve enerji kaybına neden olmaktadır. Geliştirilen sistem, mevcut ve tahmin edilen doluluk bilgilerini birlikte değerlendirerek sürücünün daha uygun park alanlarına yönlendirilmesini sağlamaktadır.

Proje kapsamında gerçekleştirilen deneylerde, random baseline yaklaşımına kıyasla yaklaşık %86,83 oranında trafik azaltma potansiyeli (proxy metric) gözlemlenmiştir. Bu sonuç, ajan tabanlı yönlendirme sisteminin rastgele park arama davranışına göre daha verimli kararlar üretebildiğini göstermektedir.

Bunun yanı sıra sistem, yalnızca anlık doluluk göstergesi sunan geleneksel çözümlerden farklı olarak geleceğe yönelik doluluk tahminlerini de karar mekanizmasına dahil etmektedir.

LSTM tabanlı zaman serisi tahmini ile PPO ve DQN gibi pekiştirmeli öğrenme algoritmalarının birlikte kullanılması, sistemin hem reaktif hem de proaktif yönlendirme yapabilmesini sağlamıştır.

Projeyi benzer çalışmalardan ayıran bir diğer önemli özellik ise çoklu algoritma karşılaştırmaları ve interaktif kullanıcı arayüzleridir. Streamlit tabanlı karar destek paneli sayesinde kullanıcılar; doluluk tahminlerini, RL ajan davranışlarını ve ödül mekanizmalarını görsel olarak inceleyebilmekte, böylece sistem kararları daha açıklanabilir hale gelmektedir.

2.4. Hedef Kullanıcı Kitlesi

Birincil kullanıcılar: şehir içi sürücüler, otopark işletmecileri, akıllı şehir yönetim birimleri. İkincil kullanıcılar: veri bilimciler, BLM4502 öğrenci/jüri değerlendiricileri, demo izleyicileri. Streamlit paneli tek kullanıcı prototip olarak tasarlanmıştır; FastAPI + React mimarisi çoklu eşzamanlı istek için ölçeklenebilir altyapı sunar.

2.5. Kullanılan Teknolojiler ve Gerekçeleri

Teknoloji	Avantaj	Dezavantaj
Python	Veri bilimi ekosistemi, okunabilirlik, ekip işbirliği	Yorumlama hızı C++'a göre düşük
PyTorch	LSTM/GRU/Transformer eğitimi, dinamik graf	TensorFlow'a göre farklı API
Stable-Baselines3	PPO/DQN hazır implementasyon, VecNormalize	Özel ortam entegrasyonu gerekir
Gymnasium	Standart RL ortam API'si	Simülasyon gerçek trafik değildir
Pandas/NumPy	Veri manipülasyonu, vektör işlemleri	Büyük veride bellek baskısı
Scikit-learn	Scaler, RF baseline, metrikler	Derin öğrenme değildir

Matplotlib	EDA ve RL grafikleri	İnteraktif görselleştirme sınırlı
Streamlit	Hızlı ML dashboard prototipi	Kurumsal UI esnekliği düşük
FastAPI	REST API, otomatik OpenAPI	Ek deployment yapılandırması
React (Vite)	Modern SPA ön uç	Node.js bağımlılığı

TABLO 2.5. Kullanılan Teknolojilerin Avantaj ve Dezavantaj Analizi

Python, veri işleme, model eğitimi ve değerlendirme süreçlerinin tamamında ortak programlama dili olarak tercih edilmiştir. Veri hazırlama (`data_preparation.py`), zaman serisi tahmini (`train_lstm.py`), pekiştirmeli öğrenme eğitimi (`train_rl.py`) ve performans değerlendirme (`evaluate.py`) modülleri aynı Python sanal ortamı içerisinde çalışacak şekilde geliştirilmiştir. Bu yaklaşım, proje bileşenleri arasında uyumluluk ve sürdürülebilirlik sağlamıştır.

Derin öğrenme tarafında TensorFlow/Keras yerine PyTorch tercih edilmiştir[11]. PyTorch'un dinamik hesaplama grafiği yapısı, modüler model geliştirme yaklaşımı ve araştırma odaklı esnekliği; özellikle LSTM tabanlı zaman serisi tahmini ve özel RL ortam entegrasyonlarında geliştirme sürecini kolaylaştırmıştır. PPO ve DQN algoritmalarının uygulanmasında ise Stable-Baselines3 kütüphanesi kullanılmıştır [6].

Projede kullanılan tüm bağımlılıklar ve sürüm gereksinimleri `requirements.txt` dosyasında tanımlanmış olup, sistemin farklı ortamlarda tekrar üretilebilir şekilde çalıştırılması hedeflenmiştir.

2.6. Destek Alınan Ancak Projede Yer Almayan Teknolojiler

- **Raspberry Pi / IoT kenar cihaz** — bütçe nedeniyle canlı veri toplama yerine Birmingham verisi.
- **Gerçek zamanlı trafik API** — veri setinde trafik akışı bulunmamaktadır.
- **ROS / SUMO simülatörü** — grid tabanlı özel ortam tercih edilmiştir.

3. TASARIM

3.1. Genel Sistem Mimarisi

Akıllı Park sistemi, veri işleme, tahminleme, karar verme ve kullanıcı etkileşimi süreçlerini kapsayan katmanlı bir mimari üzerine kurulmuştur. Sistem genel olarak beş ana katmandan oluşmaktadır: (1) Veri katmanı, (2) Tahmin katmanı, (3) Pekiştirmeli öğrenme simülasyon katmanı, (4) Değerlendirme katmanı ve (5) Sunum katmanı.

Veri katmanında ham otopark verileri işlenmekte ve model eğitime uygun hale getirilmektedir. Bu katman; ham CSV dosyaları, işlenmiş parquet dosyaları ve eğitim-test bölünmüş veri kümelerini içermektedir. Veri temizleme, özellik mühendisliği ve ölçekleme işlemleri bu aşamada gerçekleştirilmektedir.

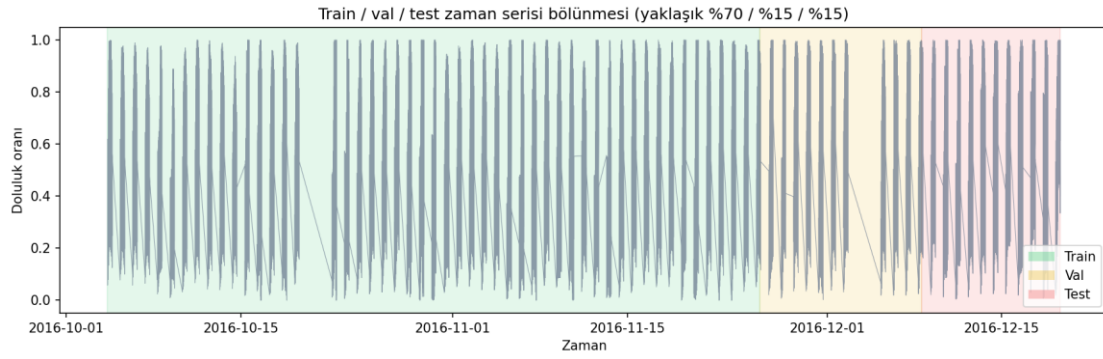
Tahmin katmanında kısa vadeli otopark doluluk tahmini yapılmaktadır. Bu amaçla LSTM tabanlı zaman serisi modeli kullanılmış, ayrıca performans karşılaştırması için Random Forest ve XGBoost gibi tabular modeller de değerlendirilmiştir. Elde edilen tahmin sonuçları, sonraki aşamada kullanılmak üzere sistem içerisinde saklanmaktadır.

RL simülasyon katmanında Gymnasium tabanlı GridParkingNavigationEnv ortamı kullanılmaktadır. Bu katmanda PPO ve DQN algoritmaları eğitilerek sürücünün en uygun otoparka yönlendirilmesi amaçlanmıştır. LSTM modelinden elde edilen tahminler, ajanın karar verme sürecini destekleyen ek bilgi olarak gözlem uzayına dahil edilmektedir.

Değerlendirme katmanında model performansları analiz edilmekte, metrikler hesaplanmakta ve grafikler ile görsel çıktılar üretilmektedir. Eğitim sonuçları, başarı oranları, ödül değerleri ve animasyon çıktıları bu katmanda oluşturulmaktadır.

Sunum katmanı ise sistemin kullanıcılarla etkileşime geçtiği bölümdür. Streamlit tabanlı karar destek paneli, FastAPI tabanlı servis katmanı ve React tabanlı kullanıcı arayüzü bu katmanda yer almaktadır. Böylece kullanıcılar tahmin sonuçlarını, RL ajan davranışlarını ve sistem performansını görsel olarak inceleyebilmektedir.

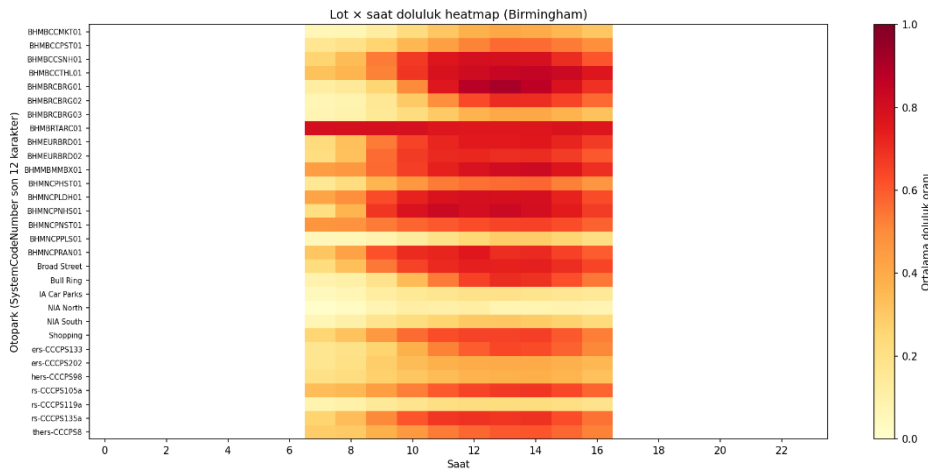
Sistem içerisindeki veri akışı tek yönlü olarak tasarlanmıştır. Ham veriler öncelikle veri işleme aşamasından geçirilmekte, ardından özellik mühendisliği ve tahmin süreçleri uygulanmaktadır. Elde edilen tahminler RL ortamına aktarılmakta, ajan eğitimi tamamlandıktan sonra sonuçlar değerlendirme ve raporlama katmanlarında analiz edilmektedir.



Şekil 5 Zaman Bazlı Train/Validation/Test Bölme Stratejisi (%70/%15/%15)

3.2. Veri Ön İşleme ve Özellik Mühendisliği

Veri hattı `data\_preparation.py` ve `utils/data\_pipeline.py` modülleri ile yürütülür. Yinelenen kayıtlar silinir; Occupancy [0, Capacity] aralığı doğrulanır; LastUpdated datetime'a dönüştürülür. Lot bazında IQR aykırı değer filtresi (çarpan=3.0, minimum 30 örnek/lot) uygulanır. Lag özellikleri (1, 3, 6, 12, 24 adım), rolling mean/variance (7 ve 24 pencere), saat, gün, hafta içi/sonu ve mevsimsel sin/cos özellikleri üretilir. MinMaxScaler yalnızca eğitim kümesinde fit edilir.



Şekil 6 Otopark Lotları × Saat Dilimi Doluluk Isı Haritası

3.3. LSTM Pipeline

Aggregate seri: tüm lotların aynı zaman damgasındaki ortalama doluluk oranı. Sliding window: LSTM_TIME_STEP=12 ile 12 ardışık zaman adımı giriş, bir sonraki adım hedef. Mimari: 2 katmanlı LSTM (hidden=64), dropout=0.2, linear head. [3] Eğitim: Adam optimizer, ReduceLROnPlateau, early stopping (patience=8), isteğe bağlı weighted L1 loss. Optuna ile hiperparametre araması desteklenir.

Parametre	Değer
Zaman penceresi (time_step)	12
Epoch (varsayılan)	80
Batch size	32
Gizli birim	64
Katman sayısı	2
Dropout	0.2
Optimizer	Adam
Kayıp fonksiyonu	L1 (weighted opsiyonel)

TABLO 3.3. LSTM Model Hiperparametreleri

3.4. RL Pipeline ve PPO Eğitim Mantığı

Her benzersiz LastUpdated zaman damgası bir RL bölümü oluşturur. ``build_grid_nav_episode_configs`` fonksiyonu lot koordinatlarını, doluluk oranlarını, hedef hücreyi (en düşük doluluklu lot) ve LSTM skalerlerini paketler. PPO eğitimi ``train_rl.py`` içinde `MlpPolicy`, `learning_rate=2.5e-4`, `n_steps=2048`, `batch_size=128`, `gamma=0.99`, `clip_range=0.2` ile yürütülür. 280.000 toplam timestep hedeflenir; Monitor CSV'ler ``logs/monitor/`` altında saklanır [4].

3.5. Durum Uzayı (State Space)

Gözlem boyutu: $5 \times 15 \times 15 + 2 = 1127$ (float32, [0,1]). Kanallar: (1) duvar maskesi, (2) park alanı maskesi, (3) hedef hücresi, (4) ajan konumu, (5) doluluk ısı haritası. Skalerler: lstm_aggregate_pred (LSTM tahmini), predicted_emptying_norm (hedef lot doluluğu vekili). Bu yapı künyede tanımlanan hibrit RL durum kodlamasını gerçekleştirir.

3.6. Eylem Uzayı (Action Space)

Discrete(4): 0=yukarı, 1=aşağı, 2=sol, 3=sağ. Duvar veya ızgara sınırına çarpışmada ajan yerinde kalır ve ek ceza alır. Geçersiz eylem indeksi yalnızca adım maliyeti uygular. Geleneksel action masking kullanılmamıştır; bunun yerine çarpışma cezası ve Manhattan shaping ile yönlendirme sağlanır.

3.7. Ödül Fonksiyonu ve Reward Shaping

Bileşen	Sabit	Açıklama
Zaman cezası	GRID_STEP_COST = 1.25	Her adımda -1.25
Manhattan shaping	SCALE = 4.5	Hedefe yaklaşıncaya pozitif
İlk ziyaret bonusu	+0.08	Yeni hücre keşfi
Tekrar ziyaret cezası	-4.0	Gereksiz dolaşım
Salınım cezası	GRID_LOOP_PENALTY = -6.0	8 adımlık A↔B döngü
Hedef bonusu	+300.0	Terminal başarı
Timeout cezası	-200.0	200 adım sınırı
Yol verimliliği	SCALE = 45.0	BFS optimaline yakınlık
Yüksek doluluk	threshold=0.88, ceza=22	Dolu lot geçişi
Ödül kırpması	GRID_REWARD_CLIP = 550	Stabilizasyon

TABLO 3.7. Ödül Fonksiyonu Bileşenleri (ml_config.py / reward_utils.py)

3.8. Grid Environment ve Dynamic Occupancy

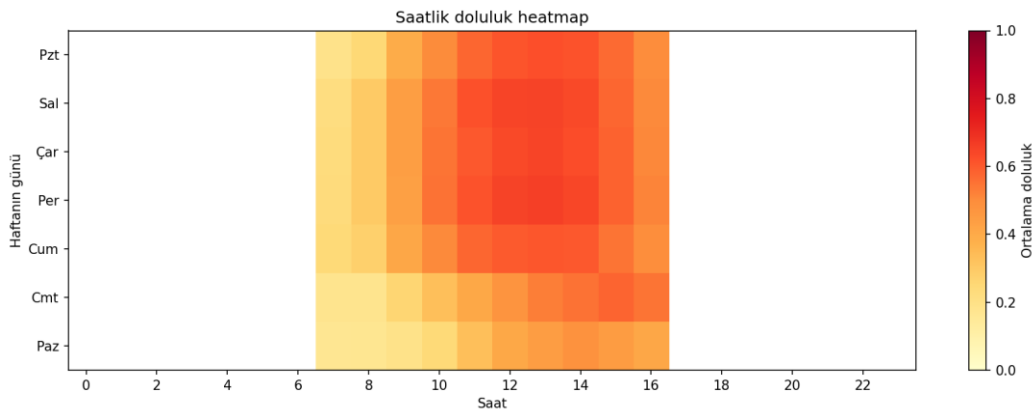
GridParkingNavigationEnv 15×15 boyutundadır; duvar hücreleri sabittir, park lotları doluluk oranına göre renklendirilir. Senaryo parametresi (`scenario`): low (×0.55), medium (×1.0), high (×1.35), dynamic (rastgele güncelleme). Eğitimde statik hedef lot (bölüm başında belirlenen en düşük doluluklu lot) tercih edilmiştir. Dynamic goal senaryoları kararlılığı düşürebilir; bu nedenle ana deneyler sabit hedef yapılandırması ile yürütülmüştür[9].

3.9. Salınım Problemi ve Stabilizasyon

Grid ortamlarında ajanların A↔B zigzag hareketi (oscillation) yaygın bir sorundur. Projede GRID_LOOP_WINDOW=8 penceresinde tekrarlayan konum deseni tespit edilerek GRID_LOOP_PENALTY=-6.0 uygulanır. Tekrar ziyaret cezası (-4.0) ve düşük ilk ziyaret bonusu (0.08) birlikte gereksiz keşfi sınırlar. PPO eğitiminde ent_coef=0.0 ayarlanarak rastgele keşif azaltılmış; VecNormalize ile ödül ölçeklemesi stabilize edilmiştir. Monitor loglarında PPO'nun son bölümlerde +300 ile +360 arası ödül ve 4–18 adım uzunluğu gözlemlenmiştir.

3.10.Explainable AI ve Streamlit Dashboard

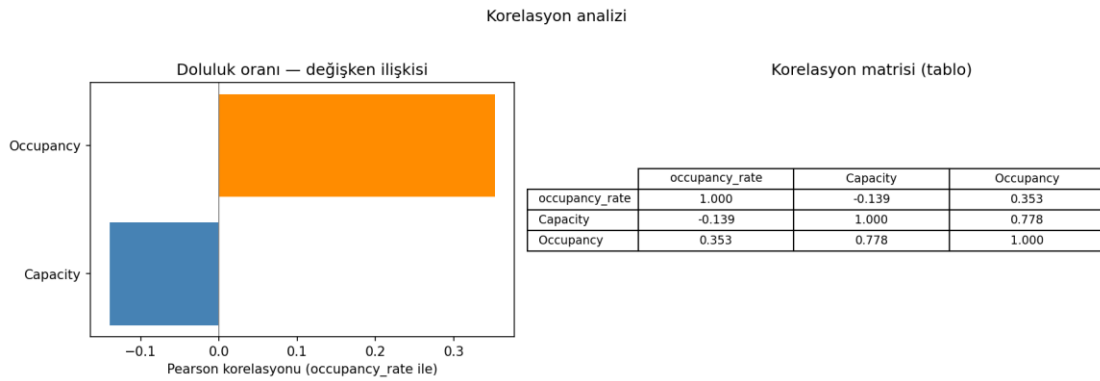
Streamlit uygulaması (`ui\_streamlit/app.py`) beş sekmeden oluşur: (1) Zaman serisi tahmin ve MAE/RMSE/MAPE/R² metrikleri, (2) EDA grafikleri, (3) Sistem özeti dashboard, (4) RL simülasyon ve adım adım grid görselleştirme, (5) Karar destek ve açıklanabilir öneri. `\_explain\_reward` fonksiyonu her adımın ödülünü Türkçe açıklamaya dönüştürür. GIF çıktıları (`ppo\_agent\_explained.gif`) canlı metrik paneli ile birlikte sunulur.



Şekil 7 Gün × Saat Doluluk Isı Haritası (RL senaryo analizi için)

3.11. Veri Akış Diyagramı ve Sequence Generation

LSTM sequence generation `utils/lstm\_core.py` modülündeki `make\_sequences` fonksiyonu ile gerçekleştirilir. Ardışık 12 zaman adımı özellik matrisi giriş; bir sonraki aggregate occupancy_rate hedef olarak atanır. RL tarafında her timestamp için `build\_grid\_nav\_episode\_configs` çağrılır; lot koordinatları `stable\_parking\_coordinates` ile hash tabanlı yerleştirilir. Tahmin CSV varsa LSTM skaleri bölüme enjekte edilir.



Şekil 8 Özellikler Arası Korelasyon Matrisi

3.12. Static Goal vs Dynamic Goal Tartışması

Dynamic goal senaryosunda hedef lot bölüm ortasında değişebilir; bu durum Manhattan shaping sinyalini zayıflatır ve ajanın osilasyon yapma eğilimini artırır. Static training yaklaşımında hedef bölüm başında belirlenir ve sabit kalır; PPO eğitim loglarında bu yapılandırmanın daha monoton artan reward trendi ürettiği gözlemlenmiştir. Dynamic senaryo (`SCENARIO\_DYNAMIC`) stres testi olarak `scenario=` parametresi ile desteklenir; ana metrikler static hedef konfigürasyonu üzerinden raporlanmıştır.

3.13.Action Masking ve Geçersiz Eylem Yönetimi

Projede klasik RL action masking (geçersiz eylemlerin policy softmax'ından çıkarılması) uygulanmamıştır. Bunun yerine duvar çarpışmasında ajan konumu değişmez ve `- GRID_STEP_COST - wall_penalty` cezası uygulanır. Park alanı maskesi gözlemin ikinci kanalında yer alır; ajan dolu lot hücresinden geçtiğinde `GRID_HIGH_OCC_STEP_PENALTY` devreye girer. Bu yaklaşım, maskesiz ortamlarda öğrenmeyi zorlaştırır da gerçek dünya navigasyonunda 'duvara çarpma' metaforunu korumaktadır.

3.14.DQN Eğitim Yapılandırması

Parametre	Değer
learning_rate	1e-4
buffer_size	150.000
learning_starts	2000
batch_size	64
gamma	0.99
exploration_fraction	0.25
exploration_final_eps	0.05

TABLO 3.14. DQN Hiperparametreleri (train_rl.py)

3.15.SWOT Analizi

Kategori	Değerlendirme
Güçlü yönler	Hibrit DL+RL, gerçek şehir verisi, açıklanabilir ödül, çoklu UI
Zayıf yönler	Statik veri, gerçek zamanlı API yok, trafik verisi eksik
Fırsatlar	Belediye/AVM/hastane entegrasyonu, akıllı şehir projeleri
Tehditler	Veri güncelliği, sensör erişim kısıtları, genellenebilirlik

TABLO 3.15. Proje SWOT Analizi

4. GELİŞTİRME

4.1. Veri Hazırlama ve Özellik Mühendisliği Uygulaması

Bu aşamada Parking Birmingham veri seti üzerinde veri hazırlama ve özellik mühendisliği çalışmaları gerçekleştirilmiştir. Ham verilerin doğrulanması, eksik değer kontrolleri, veri kümelerinin eğitim-doğrulama-test olarak ayrılması ve model eğitiminde kullanılacak özniteliklerin oluşturulması bu bölümde yürütülmüştür. Ayrıca zaman serisi modellerinin geçmiş eğilimleri öğrenebilmesi amacıyla gecikmeli (lag) özellikler üretilmiş ve veriler model eğitimine uygun hale getirilmiştir.

```
47 def validate_occupancy(df: pd.DataFrame) -> pd.DataFrame:
48     """
49     Occupancy fiziksel olarak mantıklı olsun.
50     Capacity 0 veya negatif satırlar elenir; Occupancy [0, Capacity] aralığına çekilmez,
51     aralık dışındakiler silinir (veri hatası varsayılır).
52     """
53     before = len(df)
54     cap = pd.to_numeric(df["Capacity"], errors="coerce")
55     occ = pd.to_numeric(df["Occupancy"], errors="coerce")
56     df = df.copy()
57     df["Capacity"] = cap
58     df["Occupancy"] = occ
59
60     df = df.dropna(subset=["Capacity", "Occupancy"])
61     df = df[df["Capacity"] > 0]
62     df = df[(df["Occupancy"] >= 0) & (df["Occupancy"] <= df["Capacity"])]
63
64     print(f"[data_prep] Occupancy doğrulama: {before} -> {len(df)} satır")
65     return df
```

Şekil 9 Otopark Doluluk Verisi Doğrulama ve Temizleme

Şekil 9’da fonksiyonda Capacity ve Occupancy sütunları sayısal formata çevrilip geçersiz değerler ayıklanmıştır. Ardından kapasitesi 0’dan küçük/eşit olan, doluluğu negatif olan veya doluluğu kapasiteyi aşan kayıtlar silinerek analiz için tutarlı bir veri seti elde edilmiştir.

```

79 def split_by_timestamps(
80     df: pd.DataFrame,
81     train_ratio: float = 0.70,
82     val_ratio: float = 0.15,
83 ) -> tuple[pd.DataFrame, pd.DataFrame]:
84     """
85     Zaman serisini bozmadan böler: önce benzersiz LastUpdated sırasına,
86     sonra her satır kendi zaman damgasına göre tek bir split'e düşer.
87
88     Not: Satır sayısına göre değil, benzersiz zaman sayısına göre oran uygulanır;
89     böylece aynı anda çekilmiş çoklu otopark okumaları hep aynı parçada kalır.
90     """
91     unique_times = df["LastUpdated"].drop_duplicates().sort_values(kind="mergesort")
92     n_t = len(unique_times)
93     if n_t < 3:
94         raise ValueError(f"En az 3 benzersiz zaman gerekli (train/val/test), gelen: {n_t}")
95
96     # Kesme indeksleri: Önce oransal, sonra boş küme kalmaması için sıkıştır
97     idx_train = int(n_t * train_ratio)
98     idx_val_end = int(n_t * (train_ratio + val_ratio))
99     idx_train = max(1, min(idx_train, n_t - 2))
100    idx_val_end = max(idx_train + 1, min(idx_val_end, n_t - 1))
101
102    train_times = set[Any](unique_times.iloc[:idx_train])
103    val_times = set[Any](unique_times.iloc[idx_train:idx_val_end])
104    test_times = set[Any](unique_times.iloc[idx_val_end:])
105
106    if not test_times:
107        # Çok seyrek veride son zamanı teste al
108        last_t = unique_times.iloc[-1]
109        test_times = {last_t}
110        val_times.discard(last_t)
111        train_times.discard(last_t)
112
113    train_df = df[df["LastUpdated"].isin(train_times)].copy()
114    val_df = df[df["LastUpdated"].isin(val_times)].copy()
115    test_df = df[df["LastUpdated"].isin(test_times)].copy()
116
117    print(
118        f"[data_prep] Zaman bazlı split - benzersiz zaman: {n_t}, "
119        f"train/val/test zaman sayıları: {len(train_times)}/{len(val_times)}/{len(test_times)}"
120    )

```

Şekil 10 Zaman Serisine Uygun Train/Validation/Test Bölme

Şekil 10'da veri, zaman sırası bozulmadan LastUpdated alanına göre train/validation/test kümelerine ayrılmıştır. Benzersiz zaman damgaları üzerinden bölme yapılarak aynı ana ait kayıtların farklı kümelere dağılması engellenmiş, böylece veri sızıntısı riski azaltılıp model değerlendirmesi daha güvenilir hale getirilmiştir.

```

40 LAG_STEPS = (1, 3, 6, 12, 24)
41 ROLL_WINDOWS = (7, 24)
42
43 FEATURE_COLS_FOR_SCALING = [
44     "occupancy_rate",
45     "lag_1",
46     "lag_3",
47     "lag_6",
48     "lag_12",
49     "lag_24",
50     "roll_mean_7",
51     "roll_mean_24",
52     "roll_var_7",
53     "roll_var_24",
54     "hour",
55     "day_of_week",
56     "is_weekend",
57     "month_sin",
58     "month_cos",
59     "day_of_year_sin",
60     "day_of_year_cos",
61 ]

```

Şekil 11 Özellik Mühendisliği ve Ölçeklenecek Değişkenlerin Tanımlanması

Şekil 11’de modelin geçmiş ve dönemsel örüntüleri öğrenebilmesi için gecikmeli (lag), hareketli ortalama/varyans (roll) ve takvim tabanlı zaman özellikleri bir araya getirilmiştir. Ayrıca FEATURE_COLS_FOR_SCALING listesi ile hangi sayısal değişkenlerin standartlaştırılacağı açıkça belirlenerek eğitim öncesi tutarlı bir ön işleme adımı oluşturulmuştur.

```

84 def _split_masks_by_timestamp(
85     timestamps: pd.Series,
86     train_ratio: float = 0.70,
87     val_ratio: float = 0.15,
88 ) -> pd.Series:
89     """Her satıra train/val/test etiketi; kesimler benzersiz zaman ekseninde (veri sızıntısı yok)."""
90     unique_times = timestamps.drop_duplicates().sort_values(kind="mergesort")
91     n_t = len(unique_times)
92     if n_t < 3:
93         raise ValueError(f"En az 3 benzersiz zaman gerekli, gelen: {n_t}")
94
95     idx_train = int(n_t * train_ratio)
96     idx_val_end = int(n_t * (train_ratio + val_ratio))
97     idx_train = max(1, min(idx_train, n_t - 2))
98     idx_val_end = max(idx_train + 1, min(idx_val_end, n_t - 1))
99
100     train_times = set[Any](unique_times.iloc[:idx_train])
101     val_times = set[Any](unique_times.iloc[idx_train:idx_val_end])
102     test_times = set[Any](unique_times.iloc[idx_val_end:])
103
104     if not test_times:
105         last_t = unique_times.iloc[-1]
106         test_times = {last_t}
107         val_times.discard(last_t)
108         train_times.discard(last_t)
109
110     def _label(ts: pd.Timestamp) -> str:
111         if ts in train_times:
112             return "train"
113         if ts in val_times:
114             return "val"
115         if ts in test_times:
116             return "test"
117         return "train"

```

Şekil 12 Zaman Damgasına Göre Sızıntısız Veri Etiketleme

Şekil 12’de fonksiyon, benzersiz zaman damgalarını kronolojik sırada train/validation/test aralıklarına bölerek her kayda uygun küme etiketini atar. Aynı zaman damgasına sahip örneklerin tek bir kümede kalması sağlandığı için veri sızıntısı önlenir ve modelin değerlendirme sonuçları daha güvenilir olur.

```

235 def build_processed_dataset(
236     raw_path: Path | None = None,
237     output_path: Path | None = None,
238     scaler_kind: ScalerKind | None = None,
239     outlier_filter: bool | None = None,
240 ) -> pd.DataFrame:
241     ensure_all_standard_dirs()
242     raw_path = raw_path or (DATA_RAW / "parking.csv")
243     output_path = output_path or (DATA_PROCESSED / "processed.parquet")
244
245     if not raw_path.exists():
246         raise FileNotFoundError(f"Ham veri yok: {raw_path}")
247
248     df = _load_raw(raw_path)
249     df = apply_outlier_filter(df, enabled=outlier_filter)
250     df["split"] = _split_masks_by_timestamp(df["LastUpdated"])
251     df = _add_per_lot_history_features(df)
252
253     sk = scaler_kind or str(FEATURE_SCALER).strip().lower()
254     if sk not in ("minmax", "standard"):
255         raise ValueError(f"Geçersiz scaler_kind: {sk!r} (minmax | standard)")
256     out, _ = _scale_train_only(df, MODELS_DIR, sk) # type: ignore[arg-type]
257
258     output_path.parent.mkdir(parents=True, exist_ok=True)
259     out.to_parquet(output_path, index=False)
260     print(f"[pipeline] Parquet yazıldı: {output_path} ({len(out)} satır)")
261     return out

```

Şekil 13 Uçtan Uca İşlenmiş Veri Seti Oluşturma Pipeline'ı

Şekil 13'te fonksiyon ham veriyi okuyup aykırı değer filtreleme, zaman bazlı train/validation/test etiketleme, geçmişe dayalı özellik üretimi ve ölçekleme adımlarını tek akışta uygular. Son aşamada işlenmiş veri parquet formatında kaydedilerek modelleme için tekrar üretilebilir ve standart bir veri seti hazırlanmış olur.

Bu aşama sonucunda ham otopark verileri, hem zaman serisi tahmin modelleri hem de pekiştirmeli öğrenme ortamı tarafından kullanılabilecek standart bir yapıya dönüştürülmüştür. Elde edilen veri altyapısı, sonraki geliştirme aşamalarında gerçekleştirilen model eğitimlerinin temelini oluşturmuştur.

4.2.LSTM Tabanlı Doluluk Tahmin Modelinin Gerçekleştirilmesi

Bu aşamada otopark doluluk oranlarının kısa vadeli tahmini için PyTorch tabanlı LSTM modeli geliştirilmiştir. Model, geçmiş zaman adımlarına ait doluluk verilerini kullanarak bir sonraki zaman dilimindeki doluluk oranını tahmin edecek şekilde tasarlanmıştır. Çok katmanlı LSTM mimarisi, doğrusal çıkış katmanı ve düzenleme mekanizmaları kullanılarak modelin genelleme yeteneği artırılmıştır. Ayrıca eğitim sürecinde öğrenme oranı kontrolü ve erken durdurma tekniklerinden yararlanılarak aşırı öğrenmenin önüne geçilmesi amaçlanmıştır.

```
15 class OccupancyLSTM(nn.Module):
16     """Çok katmanlı LSTM + dropout + doğrusal başlık."""
17
18     def __init__(
19         self,
20         input_dim: int,
21         hidden: int = 64,
22         num_layers: int = 2,
23         dropout: float = 0.2,
24     ):
25         super().__init__()
26         self.rnn_type = "lstm"
27         self.lstm = nn.LSTM(
28             input_dim,
29             hidden,
30             num_layers,
31             batch_first=True,
32             dropout=dropout if num_layers > 1 else 0.0,
33         )
34         self.dropout = nn.Dropout(dropout)
35         self.head = nn.Linear(hidden, 1)
36
37     def forward(self, x: torch.Tensor) -> torch.Tensor:
38         y, _ = self.lstm(x)
39         last = self.dropout(y[:, -1, :])
40         return self.head(last).squeeze(-1)
```

Şekil 14 Çok Katmanlı LSTM ile Doluluk Tahmin Modeli

Şekil 14’te sınıfta, zaman serisi doluluk verisini öğrenmek için çok katmanlı bir LSTM mimarisi tanımlanmıştır. LSTM çıkışına dropout uygulanıp son zaman adımı doğrusal katmana verilerek tek değerlik doluluk tahmini üretilmiştir.

```

53 def _train_loop(
54     model: nn.Module,
55     device: torch.device,
56     train_loader: DataLoader,
57     X_val_t: torch.Tensor,
58     y_val_t: torch.Tensor,
59     epochs: int,
60     patience: int,
61     lr: float,
62     weighted_loss: bool,
63     log_prefix: str = "[LSTM]",
64 ) -> tuple[dict[str, torch.Tensor], float]:
65     opt = torch.optim.Adam(model.parameters(), lr=lr)
66     scheduler = torch.optim.lr_scheduler.ReduceLROnPlateau(
67         opt, mode="min", factor=0.5, patience=3, min_lr=1e-6
68     )
69     loss_fn = nn.L1Loss()
70
71     best_val = float("inf")
72     stale = 0
73     best_state: dict[str, torch.Tensor] | None = None
74
75     for epoch in range(epochs):
76         model.train()
77         for xb, yb in train_loader:
78             xb = xb.to(device)
79             yb = yb.to(device)
80             opt.zero_grad()
81             pred = model(xb)
82             if weighted_loss:
83                 loss = _batch_weighted_l1(pred, yb, xb)
84             else:
85                 loss = loss_fn(pred, yb)
86             loss.backward()
87             opt.step()
88
89         model.eval()
90         with torch.no_grad():
91             vpred = model(X_val_t)
92             vloss = float(loss_fn(vpred, y_val_t).item())
93         scheduler.step(vloss)
94         print(f"{log_prefix} epoch {epoch + 1}/{epochs} val_mae_mm: {vloss:.6f}")
95
96         if vloss < best_val - 1e-6:
97             best_val = vloss
98             stale = 0
99             best_state = {k: v.cpu().clone() for k, v in model.state_dict().items()}
100         else:
101             stale += 1
102             if stale >= patience:
103                 print(f"{log_prefix} early stopping")
104                 break
105
106         if best_state:
107             model.load_state_dict(best_state)
108         assert best_state is not None
109     return best_state, best_val

```

Şekil 15 LSTM Eğitim Döngüsü, Doğrulama Takibi ve Erken Durdurma

Şekil 15’te model, her epoch’ta eğitim verisi üzerinde optimize edilip doğrulama kaybı ile performansı izlenecek şekilde eğitilmiştir. ReduceLROnPlateau ile öğrenme oranı doğrulama sonucuna göre düşürülmüş, en iyi model ağırlıkları saklanmış ve iyileşme durduğunda erken durdurma uygulanarak aşırı öğrenme riski azaltılmıştır.

Bu geliştirme aşamasında oluşturulan LSTM modeli, otopark doluluk eğilimlerini öğrenerek RL ortamında kullanılacak kestirimsel bilgiyi üretmiştir. Böylece sistem yalnızca mevcut doluluk durumuna değil, gelecekte oluşabilecek yoğunluk değişimlerine de tepki verebilir hale gelmiştir.

4.3.RL Ortamının Gerçekleştirilmesi

Pekiştirmeli öğrenme bileşeninin geliştirilmesi kapsamında gerçek otopark verilerinden Grid-Nav senaryoları üreten özel bir Gymnasium ortamı tasarlanmıştır. Bu ortamda her zaman dilimi bağımsız bir bölüm olarak değerlendirilmiş ve ajanın en uygun park alanına yönlendirilmesi hedeflenmiştir. Gözlem uzayı, doluluk haritaları ve LSTM tahminleri ile zenginleştirilmiş; böylece ajanın kararlarını hem mevcut hem de geleceğe yönelik bilgiler ışığında vermesi sağlanmıştır.

```

147 def build_grid_nav_episode_configs(
148     processed_parquet: Path | str,
149     predictions_csv: Optional[Path | str] = None,
150     split: Optional[str] = None,
151     max_episodes: int = 5000,
152     height: int = GRID_HEIGHT,
153     width: int = GRID_WIDTH,
154     base_seed: int = 42,
155 ) -> List[GridNavEpisodeConfig]:
156     """
157     Parquet zaman dilimlerinden duvarları, parka, park hücreleri, hedef ve başlangıç konumu üretir.
158
159     predictions_csv (opsiyonel): 'LastUpdated', 'y_pred_occupancy_rate' sütunları - LSTM
160     durumundaki lstm_aggregate_pred skalerine bağlar (künye: tahmin -> RL state).
161     """
162     path = Path(processed_parquet)
163     df_full = pd.read_parquet(path)
164     lot_ids = sorted(df_full["SystemCodeNumber"].astype(str).unique())
165     cap_median = df_full.groupby("SystemCodeNumber")["Capacity"].median().reindex(lot_ids)
166
167     pred_by_ts: dict[pd.Timestamp, float] = {}
168     if predictions_csv is not None:
169         pc = Path(predictions_csv)
170         if pc.exists():
171             pr = pd.read_csv(pc)
172             pr["LastUpdated"] = pd.to_datetime(pr["LastUpdated"], errors="coerce")
173             pr = pr.dropna(subset=["LastUpdated"])
174             pred_by_ts = {
175                 pd.Timestamp(ts): float(v)
176                 for ts, v in pr.groupby("LastUpdated")["y_pred_occupancy_rate"].first().items()
177             }
178
179     df = df_full if split is None else df_full[df_full["split"] == split].copy()
180     if df.empty:
181         raise ValueError(f"Split boş: {split!r}")
182
183     df["LastUpdated"] = pd.to_datetime(df["LastUpdated"])
184
185     walls = np.ones((height, width), dtype=bool)
186     walls[1 : height - 1, 1 : width - 1] = False
187
188     configs: List[GridNavEpisodeConfig] = []

```

```

189 rng = np.random.default_rng(base_seed)
190
191 for _ts, g in df.groupby("LastUpdated", sort=False):
192     if len(configs) >= max_episodes:
193         break
194     g = g.copy()
195     g["SystemCodeNumber"] = g["SystemCodeNumber"].astype(str)
196     idx = g.set_index("SystemCodeNumber")
197     cap = idx["Capacity"].astype(float).reindex(lot_ids)
198     occ = idx["Occupancy"].astype(float).reindex(lot_ids)
199     cap = cap.fillna(cap_median)
200     if cap.isna().any():
201         continue
202     occ = occ.fillna(cap)
203     mask = np.array([1.0 if lid in idx.index else 0.0 for lid in lot_ids], dtype=np.float32)
204     occ_ratio = (occ / cap).to_numpy(dtype=np.float32)
205     occ_ratio = np.clip(occ_ratio, 0.0, 1.0)
206
207     cell_map = _place_lots_on_grid(lot_ids, height, width)
208     parking_cells = [cell_map[lid] for lid in lot_ids]
209
210     valid_lot_idx = [i for i in range(len(lot_ids)) if mask[i] > 0.5]
211     if not valid_lot_idx:
212         continue
213     best_i = int(min(valid_lot_idx, key=lambda i: float(occ_ratio[i])))
214     goal_cell = cell_map[lot_ids[best_i]]
215
216     ts_key = pd.Timestamp(_ts)
217     raw_lstm = pred_by_ts.get(ts_key, float("nan"))
218     if not math.isnan(raw_lstm):
219         raw_lstm = float(np.clip(raw_lstm, 0.0, 1.0))
220     predicted_emptying_norm = float(np.clip(float(occ_ratio[best_i]), 0.0, 1.0))
221     occ_ratios_f = occ_ratio.astype(np.float32, copy=False)
222
223     blocked: Set[Tuple[int, int]] = set([Tuple[int, int]](parking_cells))
224     blocked.discard(goal_cell)
225     free_cells = [
226         (r, c)
227         for r in range(1, height - 1)
228         for c in range(1, width - 1)
229         if not walls[r, c] and (r, c) not in blocked and (r, c) != goal_cell
230     ]
231     if not free_cells:
232         continue
233     agent_start = free_cells[int(rng.integers(0, len(free_cells)))]
234
235     configs.append(
236         GridNavEpisodeConfig(
237             walls=walls.copy(),
238             parking_cells=parking_cells,
239             goal_cell=goal_cell,
240             agent_start=agent_start,
241             height=height,
242             width=width,
243             occ_ratios=occ_ratios_f,
244             lstm_aggregate_pred=float(raw_lstm),
245             predicted_emptying_norm=predicted_emptying_norm,
246         )
247     )
248
249 if not configs:
250     raise ValueError("GridNav episode üretilmedi.")
251 return configs

```

Şekil 16 Grid-Nav Bölüm Konfigürasyonlarının Zaman Diliminden Üretilmesi

Şekil 16'daki fonksiyon, işlenmiş park verisini zaman damgasına göre okuyup her dilim için duvar, park hücreleri, hedef nokta ve ajan başlangıcını içeren Grid-Nav episode konfigürasyonları oluşturur. Ayrıca doluluk oranları ile (varsa) LSTM tahminlerini birleştirerek hedef seçiminde ve ödül normalizasyonunda kullanılacak sinyalleri üretir, böylece RL ortamı gerçek veriye dayalı ve tutarlı başlangıç durumlarıyla beslenir.

```

418 def _observation(self) -> np.ndarray:
419     assert self._walls is not None
420     H, W = self.height, self.width
421     wall_f = self._walls.astype(np.float32)
422     park = np.zeros((H, W), dtype=np.float32)
423     for r, c in self._parking_set:
424         park[r, c] = 1.0
425     goal = np.zeros((H, W), dtype=np.float32)
426     goal[self._goal[0], self._goal[1]] = 1.0
427     agent = np.zeros((H, W), dtype=np.float32)
428     agent[self._agent[0], self._agent[1]] = 1.0
429     occ_map = np.clip(self._occ_heatmap, 0.0, 1.0)
430     aux = np.array(
431         [float(self._lstm_agg_scalar), float(self._pred_emptying)],
432         dtype=np.float32,
433     )
434     return np.concatenate(
435         [
436             wall_f.ravel(),
437             park.ravel(),
438             goal.ravel(),
439             agent.ravel(),
440             occ_map.ravel(),
441             aux,
442         ]
443     ).astype(np.float32)

```

Şekil 17 RL Gözlem Vektörünün Çok Kanallı Oluşturulması

Şekil 17’deki fonksiyon, ortam durumunu temsil eden duvar, park alanı, hedef, ajan ve doluluk ısı haritası katmanlarını tek bir gözlem vektöründe birleştirir. Buna ek olarak LSTM’den gelen toplu tahmin ölçeği ve öngörülen boşalma sinyali de eklenerek ajan kararlarının hem mekânsal hem de kestirimsel bilgiye dayanması sağlanır.

```

516 def step(self, action: int):
517     self._steps += 1
518     self._routing_time_cost += float(self.step_cost)
519     self._apply_dynamic_occupancy_shift()
520     a = int(action)
521     self._last_action = a
522     if a < 0 or a > 3:
523         self._invalid_move_count += 1
524         obs = self._observation()
525         r = self._clip(-self.step_cost)
526         truncated = self._steps >= self.max_episode_steps
527         info: Dict[str, Any] = {"invalid": True, "success": False, "collision": False}
528         if truncated:
529             r = self._clip(r + self.timeout_penalty)
530             info["timeout"] = True
531             info["loop_penalty_events"] = self._episode_loop_events
532             info["revisit_events"] = self._episode_revisit_events
533             info["loop_count"] = self._episode_loop_events
534             info["revisit_count"] = self._episode_revisit_events
535         info = self._merge_episode_cost_metrics(info, False, truncated)
536         info = self._enrich_step_info(info, r, a)
537         return obs, r, False, truncated, info
538
539     dr, dc = DR_DC[a]
540     nr, nc = self._agent[0] + dr, self._agent[1] + dc
541     hit_wall = (
542         nr < 0
543         or nc < 0
544         or nr >= self.height
545         or nc >= self.width
546         or self._walls[nr, nc]
547     )

```

Şekil 18 RL Ortamında step() ile Eylem Uygulama ve Ceza Yönetimi

Şekil 18’de ajanın seçtiği eylem uygulanarak adım sayacı, hareket maliyeti ve dinamik doluluk durumu güncellenir. Geçersiz eylemler için anında ceza verilir, bölüm sonu koşulları ve olay sayaçları info içine işlenerek öğrenme süreci için tutarlı geri bildirim üretilir.

4.4.Ödül Sistemi ve PPO Eğitim Süreci

RL ajanının etkili öğrenebilmesi için çok bileşenli bir ödül sistemi geliştirilmiştir. Tasarlanan ödül fonksiyonu yalnızca hedefe ulaşmayı değil, aynı zamanda gereksiz dolaşımı azaltmayı, yüksek doluluklu bölgelerden kaçınmayı ve daha verimli rotalar izlemeyi teşvik etmektedir. PPO algoritması bu ödül yapısı altında eğitilmiş ve eğitim süreci boyunca performans metrikleri takip edilmiştir.

```

75 def compute_grid_step_reward(
76     *,
77     step_cost: float,
78     manhattan_scale: float,
79     old_dist: int,
80     new_dist: int,
81     was_revisit: bool,
82     first_visit_bonus: float,
83     revisit_penalty: float,
84     loop_hit: bool,
85     loop_penalty: float,
86     goal_bonus: float,
87     timeout_penalty: float,
88     terminated: bool,
89     truncated: bool,
90     reward_clip: float,
91     shortest_path_len: int,
92     path_edges: int,
93     path_efficiency_scale: float,
94 ) -> Tuple[float, RewardParts]:
95     """
96     Grid üzerinde **geçerli hücreye yapılan bir adım** için ödül.
97
98     Duvar/sırtı çarpılması ve geçersiz aksiyon indeksi ortam tarafında ayrıştırmaya
99     ('-step_cost' ve isteğe bağlı '-wall_penalty').
100     """
101     time_penalty = -float(step_cost)
102     distance_shaping = float(manhattan_scale) * float(old_dist - new_dist)
103     visit_adjustment = float(revisit_penalty if was_revisit else first_visit_bonus)
104     loop_p = float(loop_penalty) if loop_hit else 0.0
105     g_bonus = 0.0
106     t_pen = 0.0
107     path_eff_bonus = 0.0
108
109     total = time_penalty + distance_shaping + visit_adjustment + loop_p
110
111     if terminated:
112         g_bonus = float(goal_bonus)
113         total += g_bonus
114         path_eff_bonus = compute_goal_path_efficiency_bonus(
115             success=True,
116             shortest_path_len=int(shortest_path_len),
117             path_edges=int(path_edges),
118             scale=float(path_efficiency_scale),
119             reward_clip=float(reward_clip),
120         )
121         total += path_eff_bonus
122
123     if truncated and not terminated:
124         t_pen = float(timeout_penalty)
125         total += t_pen
126
127     clipped = clip_reward(total, reward_clip)
128     parts = RewardParts(
129         time_penalty=time_penalty,
130         distance_shaping=distance_shaping,
131         visit_adjustment=visit_adjustment,
132         loop_penalty=loop_p,
133         goal_bonus=g_bonus,
134         timeout_penalty=t_pen,
135         path_efficiency_bonus=path_eff_bonus,
136         wall_collision_extra=0.0,
137         clipped_total=clipped,
138     )
139     return clipped, parts

```

Şekil 19 Grid Ortamı İçin Bileşenli Ödül Hesaplama Fonksiyonu

Şekil 19'daki fonksiyon, ajan her geçerli adım attığında zaman maliyeti, hedefe yaklaşma, yeniden ziyaret/loop cezası, hedefe ulaşma bonusu ve süre aşımı cezası gibi tüm ödül bileşenlerini birlikte hesaplar. Hesaplanan toplam ödül clip edilerek stabilize edilir ve alt bileşenler ayrı ayrı döndürülerek eğitim sürecinin daha şeffaf analiz edilmesi sağlanır.

```

86 model = PPO(
87     "MlpPolicy",
88     vec,
89     verbose=1,
90     seed=RANDOM_SEED,
91     tensorboard_log=str(tb),
92     learning_rate=2.5e-4,
93     n_steps=2048,
94     batch_size=128,
95     gamma=0.99,
96     gae_lambda=0.95,
97     ent_coef=0.0,
98     clip_range=0.2,
99     n_epochs=10,
100 )

```

Şekil 20 PPO Ajanı İçin Eğitim Hiperparametrelerinin Yapılandırılması

Şekil 20’de PPO ajanı MlpPolicy ile başlatılarak öğrenme oranı, adım sayısı, batch boyutu, indirim katsayısı ve GAE gibi temel hiperparametreler tanımlanmıştır. Ayrıca tekrar üretilebilirlik için sabit tohum (seed) verilmiş ve eğitim sürecinin izlenebilmesi amacıyla TensorBoard kayıt yolu yapılandırılmıştır.

```
119     model.learn(total_timesteps=timesteps, progress_bar=True)
120     # Kayıt anında istatistikleri dondur; aksi halde değerlendirme sırasında dağılım kayar
121     vec.training = False
122     vec.norm_reward = False
123     base = MODELS_DIR / f"{algo_l}_agent"
124     model.save(str(base))
125     vn_path = MODELS_DIR / f"vecnormalize_{algo_l}.pkl"
126     vec.save(str(vn_path))
127     print(f"[RL] Model: {base}.zip")
128     print(f"[RL] VecNormalize: {vn_path}")
129     return MODELS_DIR / f"{algo_l}_agent.zip"
```

Şekil 21 RL Modelinin Eğitilip Kaydedilmesi ve Normalizasyon Parametrelerinin Dondurulması

Şekil 21’de model belirlenen toplam zaman adımı kadar eğitilir, ardından değerlendirme sırasında dağılım kaymasını önlemek için VecNormalize istatistik güncellemeleri kapatılır. Son olarak ajan ağırlıkları ve normalizasyon parametreleri ayrı dosyalara kaydedilerek modelin aynı ön işleme koşullarıyla güvenli şekilde yeniden yüklenmesi sağlanır.

```
51 | RL_TOTAL_TIMESTEPS: int = 280_000
52   GRID_GOAL_BONUS: float = 300.0
53   GRID_TIMEOUT_PENALTY: float = -200.0
54   # Hedefe yaklaşmayı adım başına güçlendirir (Manhattan d_new - d_old pozitifken).
55   GRID_MANHATTAN_SHAPING_SCALE: float = 4.5
56   GRID_STEP_COST: float = 1.25
```

Şekil 22 RL Eğitim Süresi ve Ödül- Ceza Katsayılarının Kalibrasyonu

Şekil 22’de RL eğitiminin toplam adım sayısı ile hedef bonusu, zaman aşımı cezası, Manhattan tabanlı yönlendirme ölçeği ve adım maliyeti gibi temel ödül parametreleri tanımlanmıştır. Parametreler birlikte ayarlanarak ajanın hedefe daha hızlı yaklaşması teşvik edilirken gereksiz dolaşma ve gecikme davranışları cezalandırılmıştır.

4.5.Dashboard ve Görselleştirme Araçlarının Geliştirilmesi

Projenin sonuçlarının kullanıcı tarafından kolay anlaşılabilmesi amacıyla Streamlit tabanlı bir karar destek arayüzü geliştirilmiştir [12]. Bu arayüz üzerinden zaman serisi tahmin sonuçları, EDA çıktıları, RL ajan animasyonları ve ödül bileşenleri görsel olarak sunulmaktadır. Böylece sistemin yalnızca performansı değil, karar alma süreci de açıklanabilir hale getirilmektedir.

```
45 tab_ts, tab_eda, tab_dash, tab_rl, tab_decision = st.tabs(  
46     [  
47         "1) Zaman serisi tahmin & performans",  
48         "2) Keşifsel veri analizi (EDA)",  
49         "3) Sistem özeti (dashboard)",  
50         "4) RL simülasyon & görselleştirme",  
51         "5) Karar destek & öneri",  
52     ]  
53 )
```

Şekil 23 Streamlit Arayüzünde Modüler Sekme Yapısının Oluşturulması

Şekil 23’te uygulama, farklı analiz ve kullanım senaryolarını ayırtmak için beş ana sekmeye bölünmüştür. Böylece zaman serisi tahmini, EDA, dashboard, RL simülasyonu ve karar destek çıktıları kullanıcıya düzenli ve erişilebilir bir akışla sunulmuştur.

```

72 def _explain_reward(reward: float, terminated: bool, truncated: bool) -> str:
73     """RL adımındaki reward değerini kullanarak anlaşılan şekilde yorumlar."""
74     if terminated and reward > 0:
75         return "Ajan başarılı bir sonuca ulaştı veya hedef park alanına yaklaştı."
76     if truncated:
77         return "Bölüm maksimum adım sınırına ulaştı; ajan hedefe zamanında ulaşamadı."
78     if reward > 0:
79         return "Bu adım olumlu sonuç verdi; ajan hedefe yaklaştı veya daha avantajlı duruma geçmiş olabilir."
80     if reward < -1:
81         return "Bu adım yüksek ceza aldı; çarpışma, gereksiz tekrar veya kötü yön seçimi olabilir."
82     if reward < 0:
83         return "Bu adım küçük maliyet aldı; ajan hareket ettiği için step cost uygulanmış olabilir."
84     return "Bu adım nötr etki oluşturdu."

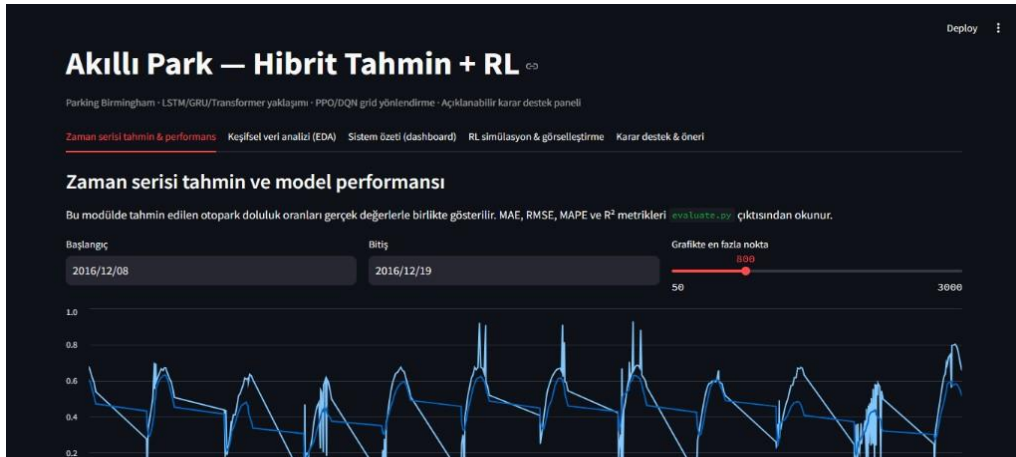
```

Şekil 24 RL Ödül Değerlerinin Açıklanması

Şekil 24’te fonksiyon, alınan ödülün büyüklüğü ve bölümün bitiş durumuna göre ajanın davranışını metinsel olarak yorumlar. Böylece ham sayısal reward değerleri, başarı, zaman aşımı, ceza veya nötr etki gibi anlamlı açıklamalara dönüştürülerek sonuçların yorumlanması kolaylaştırılır.

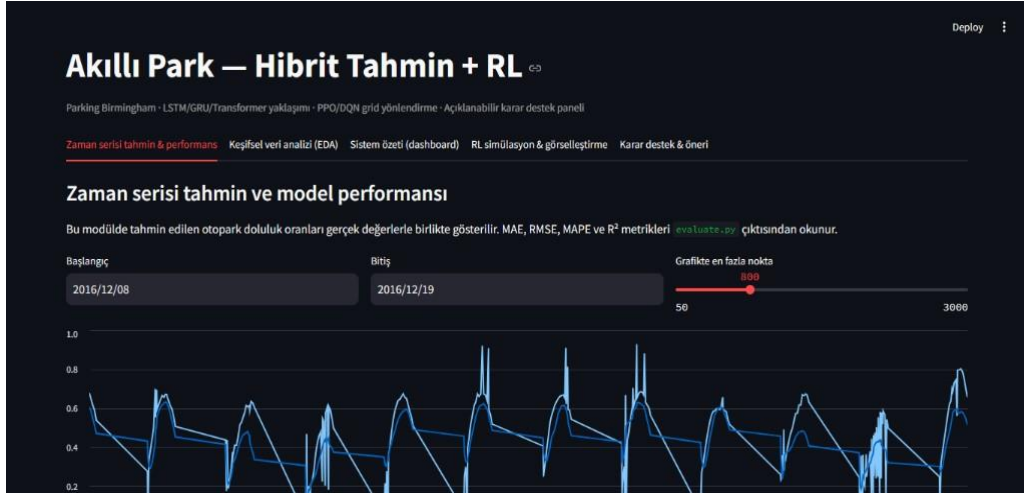
4.5.1. Streamlit Tabanlı Kullanıcı Arayüzü

Geliştirilen Streamlit paneli (`ui\_streamlit/app.py`), hibrit tahmin ve RL bileşenlerinin son kullanıcıya sunulduğu etkileşimli arayüzdür. Aşağıdaki ekran görüntüleri, sistemin beş ana sekmesi üzerinden gerçek çalışma ortamında elde edilmiştir.



Şekil 25 Akıllı Park Streamlit Arayüzü Genel Görünümü

Şekil 25'te, geliştirilen Akıllı Park uygulamasının Streamlit tabanlı ana kullanıcı arayüzünü göstermektedir. Üst bölümde proje başlığı, alt sistem bileşenlerine ilişkin kısa açıklama ve beş modüler sekmeden oluşan gezinme çubuğu yer almaktadır. Bu yapı, tahmin, analiz, özet, RL simülasyonu ve karar destek işlevlerinin tek bir panel üzerinden erişilebilir biçimde sunulmasını sağlamaktadır.



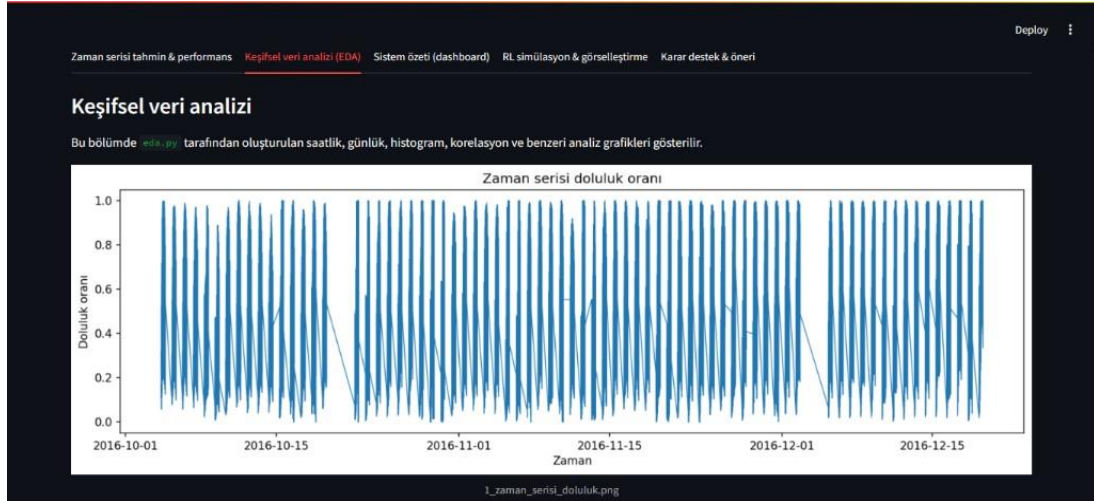
Şekil 26 Zaman Serisi Tahmin ve Model Performansı Ekranı

Zaman serisi tahmin sekmesi, kullanıcıya başlangıç–bitiş tarihi seçimi ve grafikte gösterilecek maksimum nokta sayısını ayarlama imkânı sunmaktadır. Seçilen zaman aralığında gerçek doluluk eğilimleri ile model tahminleri üst üste bindirilerek görsel karşılaştırma yapılabilir. Bu ekran, LSTM tabanlı tahmin modülünün pratik kullanım senaryosunu doğrudan yansıtmaktadır.



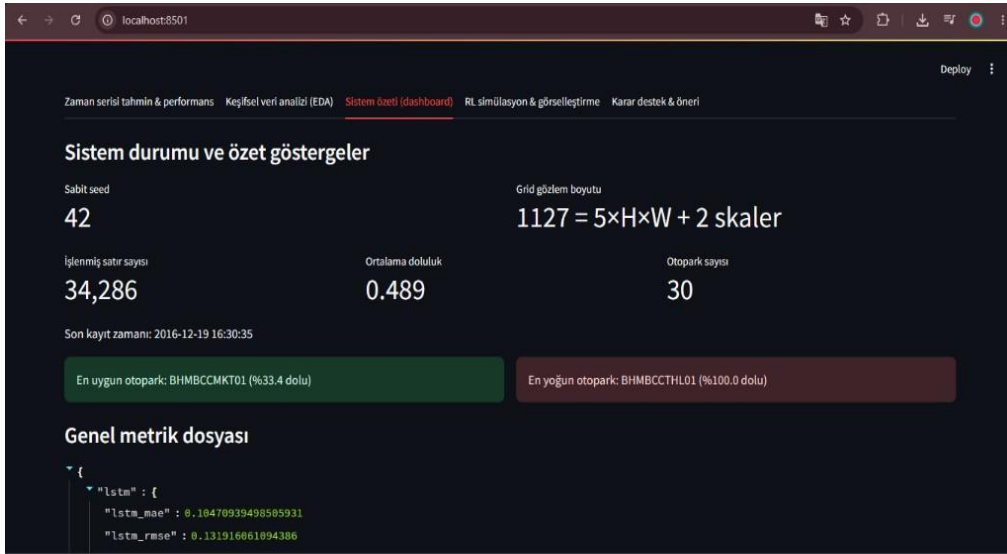
Şekil 27 Gerçek Doluluk Oranı ile Tahmin Sonuçlarının Karşılaştırılması

Şekil 27’de gerçek doluluk oranı ile model tahminleri aynı zaman eksenı üzerinde çizdirilmiş; günlük periyodik örüntüler net biçimde gözlemlenebilmektedir. Grafiğin altında MAE, RMSE, MAPE ve R^2 metrikleri anlık olarak sunulularak model performansının nicel değerlendirmesi desteklenmektedir. Elde edilen sonuçlar, tahmin bileşeninin test kümesinde kabul edilebilir hata düzeyinde çalıştığını göstermektedir.



Şekil 28 Keşifsel Veri Analizi Sekmesi

Şekil 28’de keşifsel veri analizi (EDA) sekmesi, `eda.py` betiği ile üretilen görsel çıktıların raporlanması için ayrılmıştır. Zaman serisi doluluk oranı grafiği, veri setindeki mevsimsellik, tepe saatler ve uzun dönemli eğilimler hakkında öngörü sağlamaktadır. Bu modül, model geliştirme sürecinin veri odaklı adımlarını son kullanıcıya şeffaf biçimde aktarmaktadır.



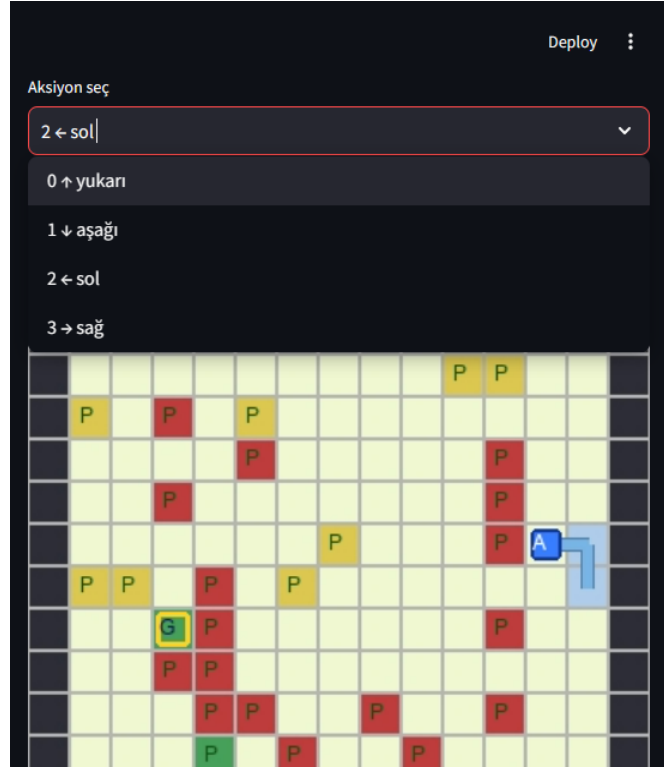
Şekil 29 Sistem Durumu ve Özet Göstergeler Dashboard Ekranı

Şekil 29’da sistem özeti ekranı, platformun genel çalışma durumunu ve veri seti istatistiklerini tek bir panel üzerinden sunmaktadır. İşlenmiş satır sayısı, otopark sayısı, ortalama doluluk ve son kayıt zamanı gibi kritik göstergelerin yanı sıra en uygun ve en yoğun otopark lotları renk kodlu olarak vurgulanmaktadır. LSTM modeline ait hata metrikleri JSON biçiminde görüntülenerek sistem bütünlüğü izlenebilir kılınmaktadır.



Şekil 30 PPO/DQN Eğitim Grafiği Arayüzü

Şekil 30’da RL eğitim grafiği modülü, kullanıcının PPO veya DQN algoritmasını seçerek eğitim sürecini görselleştirmesine olanak tanımaktadır. Bölüm ödülü ve 50 adımlık hareketli ortalama eğrisi birlikte sunularak ajanın öğrenme dinamiği izlenebilmektedir. Hareketli ortalamanın zaman içinde yükselmesi, politikanın istikrarlı biçimde iyileştiğini göstermektedir.



Şekil 31 RL Simülasyonunda Aksiyon Seçimi ve Grid Görselleştirmesi

Şekil 31’de, RL simülasyon sekmesi, ajanın 15×15 ızgara ortamındaki konumunu, hedef noktayı ve park hücrelerinin doluluk durumunu interaktif olarak görselleştirir. Kullanıcı manuel aksiyon seçimi yaparak ajanın yukarı, aşağı, sol ve sağ hareketlerini test edebilmektedir. Ajan yörüngesi yarı saydam iz ile gösterilerek karar zincirinin uzamsal takibi kolaylaştırılmaktadır.

Deploy

Ajan karar günlüğü

	adım	aksiyon	reward	karar_yorumu	terminated	truncated	info
0	1	0 + yukarı	-5.67	Bu adım yüksek ceza aldı; çarpışma, gereksiz tekrar veya kötü yön seçimi olabilir.	☐	☐	[{"action":0,"action_name":"UP","collision":false,"collis
1	2	2 + sol	3.33	Bu adım olumlu sonuç verdi; ajan hedefe yaklaşmış veya daha avantajlı duruma ge	☐	☐	[{"action":2,"action_name":"LEFT","collision":false,"coll
2	3	2 + sol	3.33	Bu adım olumlu sonuç verdi; ajan hedefe yaklaşmış veya daha avantajlı duruma ge	☐	☐	[{"action":2,"action_name":"LEFT","collision":false,"coll
3	4	1 + aşağı	3.33	Bu adım olumlu sonuç verdi; ajan hedefe yaklaşmış veya daha avantajlı duruma ge	☐	☐	[{"action":1,"action_name":"DOWN","collision":false,"cc
4	5	0 + yukarı	-9.75	Bu adım yüksek ceza aldı; çarpışma, gereksiz tekrar veya kötü yön seçimi olabilir.	☐	☐	[{"action":0,"action_name":"UP","collision":false,"collis

Son aksiyon
0 ↑ yukarı

Son reward
-9.750

Bölüm durumu
Devam ediyor

Son karar yorumu: Bu adım yüksek ceza aldı; çarpışma, gereksiz tekrar veya kötü yön seçimi olabilir.

Şekil 32 Ajan Karar Günlüğü Tablosu

Ajan karar günlüğü, her adımda seçilen aksiyon, elde edilen ödül ve otomatik üretilen karar yorumunu tablo biçiminde sunmaktadır. Olumsuz ödüller için çarpışma, gereksiz tekrar veya hatalı yön seçimi; olumlu ödüller için ise hedefe yaklaşma gibi olası nedenler metinsel olarak açıklanmaktadır. Bu yapı, pekiştirmeli öğrenme sürecinde açıklanabilirlik ve hata ayıklama kapasitesini artırmaktadır.



Şekil 33 Adım Bazlı Ödül Değişim Grafiği

Şekil 33, adım bazlı ödül grafiği, bir bölüm boyunca ajanın aldığı anlık ödül değerlerinin zaman içindeki değişimini çizgisel olarak göstermektedir. Grafiğin alt bölümünde son aksiyon, son reward ve bölüm durumu özet metrikleri yer almaktadır. Keskin düşüşler yüksek ceza alınan adımları belirginleştirerek ödül fonksiyonunun ajan davranışına etkisini analiz etmeyi mümkün kılmaktadır.



Şekil 34 Karar Destek ve Açıklanabilir Otopark Önerisi Ekranı

Şekil 34, karar destek modülü, seçilen veri dilimi ve zaman damgası için en uygun otopark lotunu doluluk oranı, boş kapasite ve uygunluk skoru ile birlikte önermektedir. Önerinin gerekçesi madde madde açıklanarak sistemin yalnızca bir sonuç üretmediği, karar sürecini şeffaf biçimde raporladığı gösterilmektedir. Bu ekran, Explainable AI bileşeninin uçtan uca kullanıcı deneyimini tamamlayan son halkasını oluşturmaktadır.

4.6.Kurulum ve Çalıştırma Yönergeleri:

Proje kök dizininde aşağıdaki adımlar izlenir:

```
python -m venv .venv && .\venv\Scripts\Activate.ps1 (Windows)
pip install -r requirements.txt
data/raw/parking.csv dosyasını yerleştirin
python data_preparation.py
python eda.py (isteğe bağlı EDA grafikleri)
python train_lstm.py --cell lstm
python train_tabular_baselines.py (isteğe bağlı)
python train_rl.py --algo both
python evaluate.py --part all
python evaluate_agents.py --episodes 40
streamlit run ui_streamlit/app.py
```

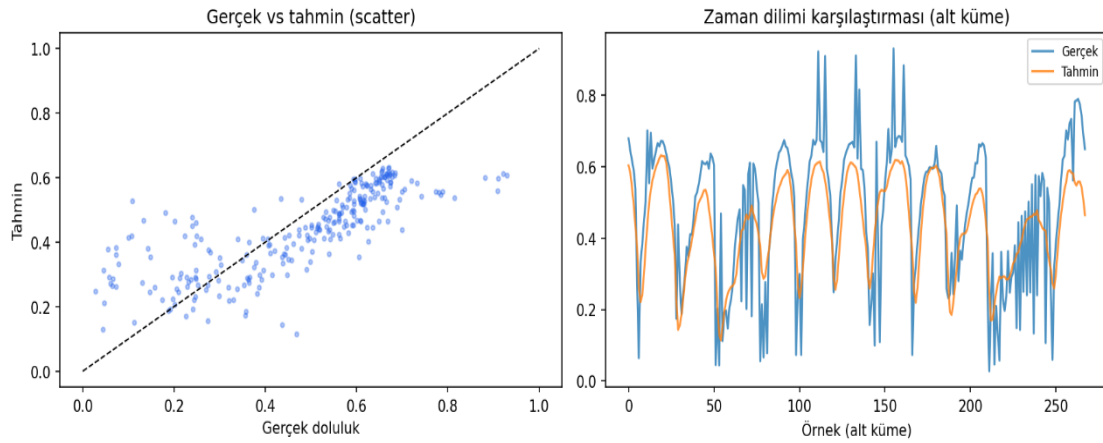
5. TEST ve DOĞRULAMA

5.1. LSTM Tahmin Performansı

Model	MAE	RMSE	MAPE (%)	R ²
LSTM (PyTorch)	0.1047	0.1319	44.93	0.5563
Random Forest	0.0807	0.1132	37.42	0.6732
XGBoost	0.0864	0.1188	38.83	0.6403

TABLO 5.1. Zaman Serisi Tahmin Modellerinin Test Kümesi Metrikleri

LSTM modeli test kümesinde **MAE=0.1047**, **RMSE=0.1319**, **MAPE=%44.93** ve **R²=0.5563** değerlerini elde etmiştir (**n=268**). Tabular modeller (RF R²=0.6732, XGB R²=0.6403) bu veri setinde LSTM'den daha yüksek R² göstermiştir; ancak LSTM çıktısı RL gözlemine entegre edilebilir sürekli tahmin sinyali sağlar. MAPE değerinin yüksek olması, düşük doluluk anlarında küçük mutlak hataların oransal olarak büyük görünmesinden kaynaklanmaktadır.



Şekil 35 LSTM Modeli Gerçek ve Tahmin Edilen Doluluk Oranı Karşılaştırması

5.2. RL Ajan Performansı

Algoritma	Başarı (%)	Ort. Ödül	Ort. Adım	Çarpışma
Random	47.5	-751.80	138.4	9.48
Greedy Manhattan	100.0	356.12	8.6	0.0
BFS Oracle	100.0	356.12	8.6	0.0
PPO	92.5	92.26	23.1	0.0
DQN	27.5	-1212.63	147.8	54.45

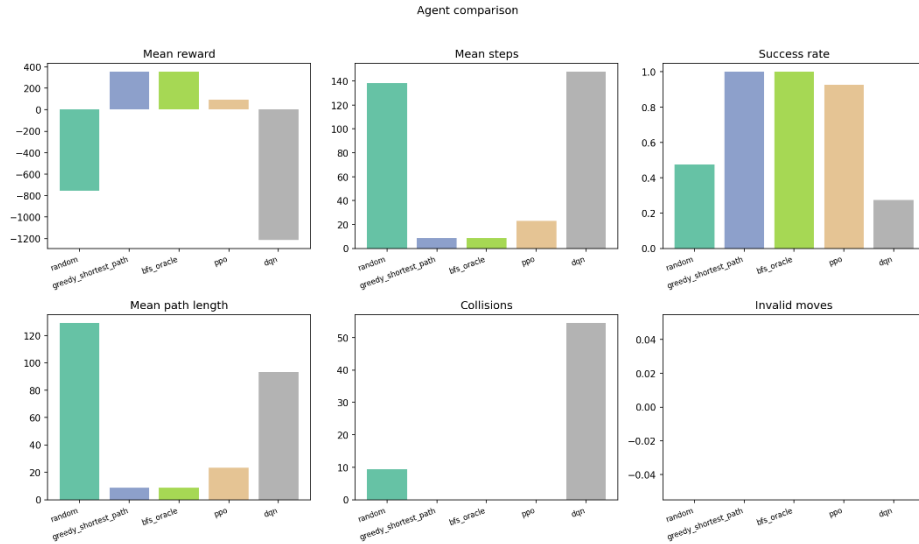
TABLO 5.2. PPO, DQN ve Baseline Algoritmalarının Karşılaştırması (40 test bölümü)

100 bölümlük değerlendirmede PPO başarı oranı **%93.0**, ortalama bölüm getirisi **169.07** ve ortalama arama adımı **24.83** olarak ölçülmüştür. Random baseline'a kıyasla trafik yoğunluğu azaltma proxy değeri **%86.83** olarak raporlanmıştır ('evaluate.py'). PPO, DQN'e göre belirgin üstünlük göstermiştir [5]: DQN başarı oranı yalnızca **%27.5** olup ortalama **54.45** duvar çarpışması yapmıştır.

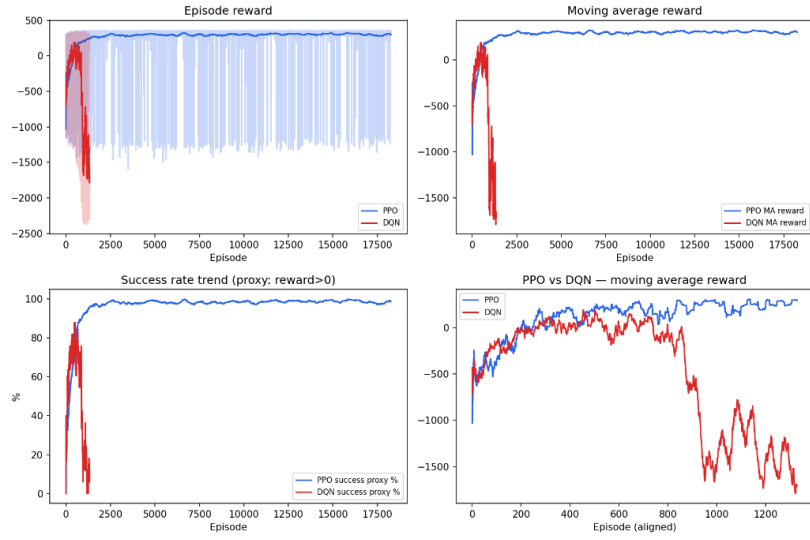


Şekil 36 PPO Ajanının Hedef Otoparka Ulaşma Performansı

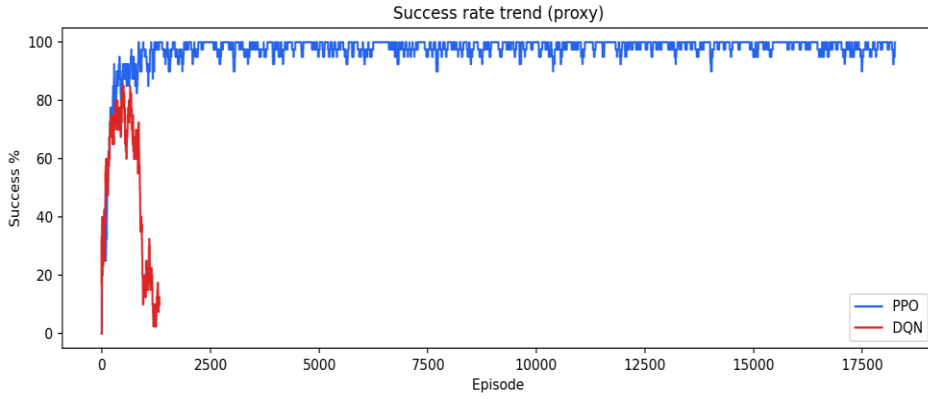
Şekil 36’da PPO ajanının park ortamında hedef otoparka ulaşmak için izlediği rota görülmektedir. Ajanın düşük doluluklu hedef bölgeye minimum sayıda adımla ulaşabildiği ve ödül şekillendirme mekanizmasından etkin şekilde yararlandığı gözlemlenmiştir.



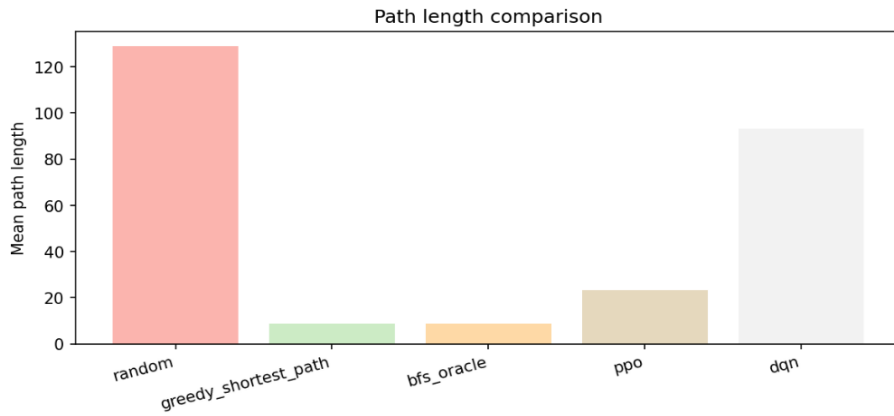
Şekil 37 PPO ve DQN Algoritmalarının Çoklu Metrik Karşılaştırması



Şekil 38 PPO ve DQN Eğitim Sürecinde Episode Reward Trend Eğrileri



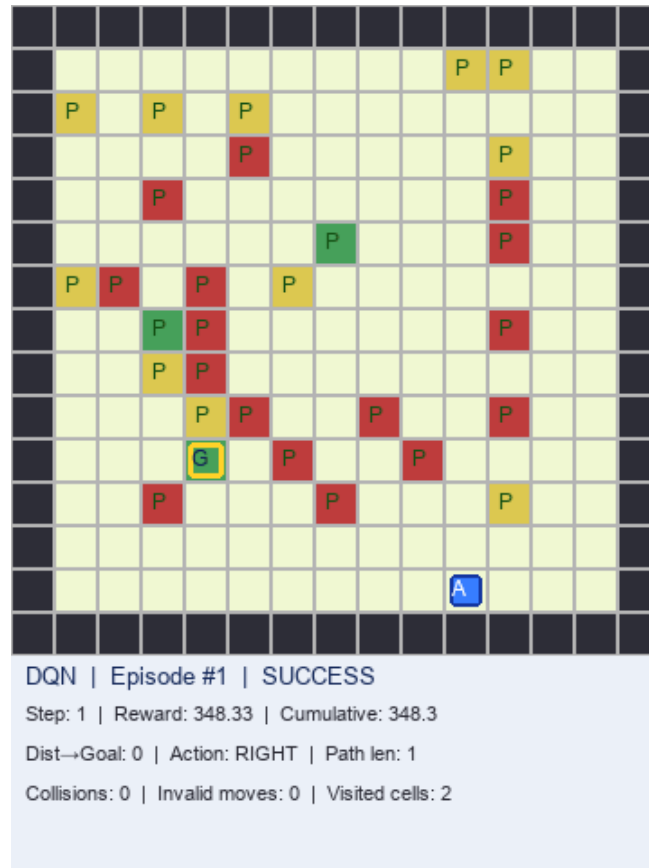
Şekil 39 Eğitim Sürecinde Başarı Oranı Proxy Trendi (ödül > 0)



Şekil 40 Algoritmaların Ortalama Rota Uzunluğu Karşılaştırması

5.3. PPO vs DQN Analizi

PPO'nun DQN'den üstün performans göstermesinin başlıca nedenleri: (1) 1127 boyutlu sürekli-benzeri gözlem uzayında policy gradient yaklaşımının daha stabil olması, (2) Monitor loglarında PPO'nun ~18.000 bölüm, DQN'in ~1.300 bölüm üretmesi — DQN episode'ları timeout nedeniyle çok daha uzun, (3) DQN'in yüksek çarpışma oranı (54.45 ortalama) öğrenmeyi bozar, (4) PPO hiperparametreleri (ent_coef=0, clip_range=0.2) bu ortam için daha uygundur.



Şekil 41 PPO ve DQN Karşılaştırmasında DQN Ajan Davranışı

Şekil 41’de DQN ajanının aynı ortam üzerindeki davranışı gösterilmektedir. PPO algoritmasına kıyasla daha uzun karar süreçleri ve daha düşük başarı oranı gözlemlenmiştir.

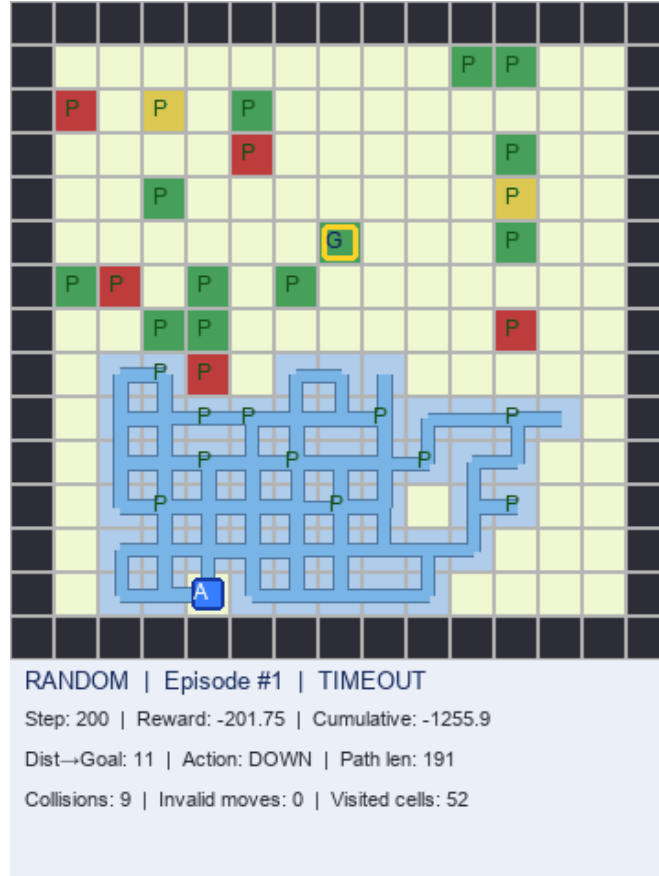
5.4. Baseline Karşılaştırması

Greedy Manhattan ve BFS oracle politikaları **%100** başarı ve **8.6** ortalama adım ile üst sınırı temsil eder. PPO ortalama **23.1** adımda **%92.5** başarı sağlayarak öğrenilmiş politikanın oracle'a makul yakınlıkta olduğunu gösterir. Random politika **%47.5** başarı ile düşük performans sergiler.



Şekil 42 Greedy Shortest Path Baseline Sonucu

Şekil 42’de Greedy Shortest Path algoritmasının hedefe ulaşmak için izlediği yol gösterilmektedir. Algoritma en kısa mesafeyi tercih etmekte ancak doluluk bilgisini dikkate almamaktadır.

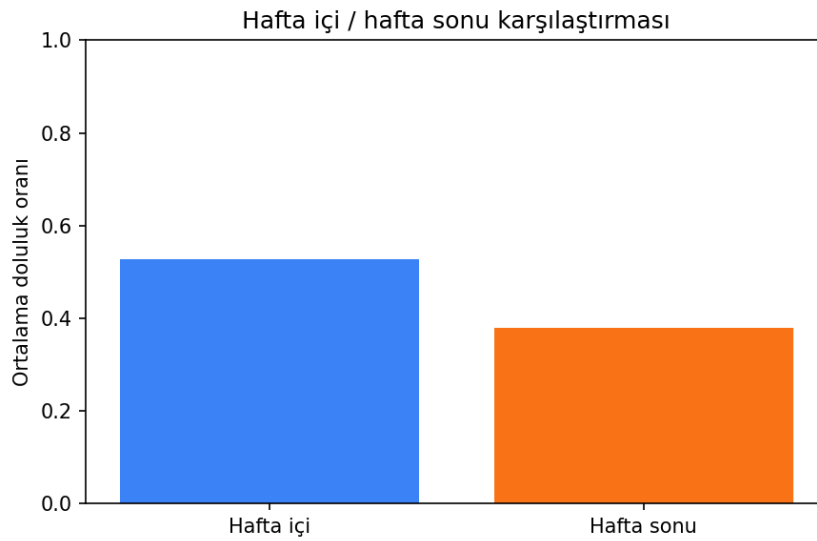


Şekil 43 Random Baseline Sonucu

Şekil 43'te rastgele hareket eden baseline ajanın davranışı gösterilmektedir. Sistematik bir strateji bulunmadığından performansı diğer yöntemlere göre düşüktür.

5.5. Oscillation ve Reward Shaping Etkisi

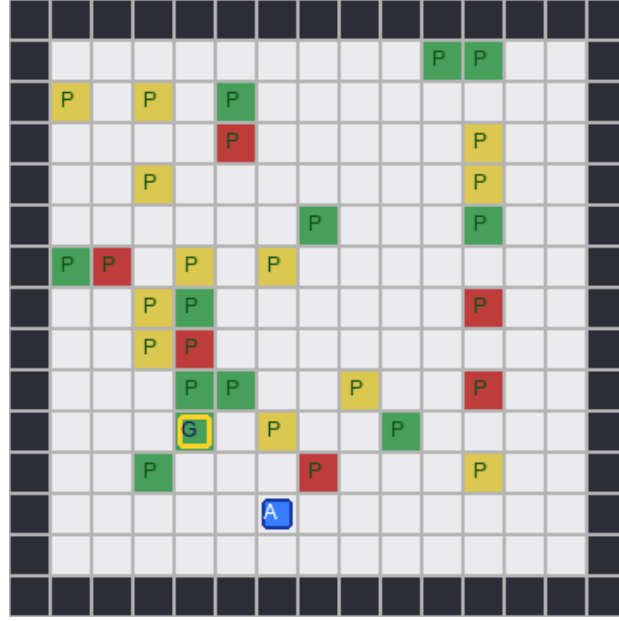
Salınım cezası (GRID_LOOP_PENALTY) ve tekrar ziyaret cezası uygulanmadan önce Pilot denemelerde ajanların zigzag hareketi gözlemlenmiştir. 8 adımlık pencere ile uygulanan döngü cezası, PPO'nun ortalama rota uzunluğunu 147.8 (DQN) yerine 23.1 seviyesine indirmiştir. Static hedef eğitimi, dynamic goal senaryolarına kıyasla daha kararlı öğrenme eğrisi üretmiştir.



Şekil 44 Hafta İçi ve Hafta Sonu Doluluk Karşılaştırması (EDA doğrulama)

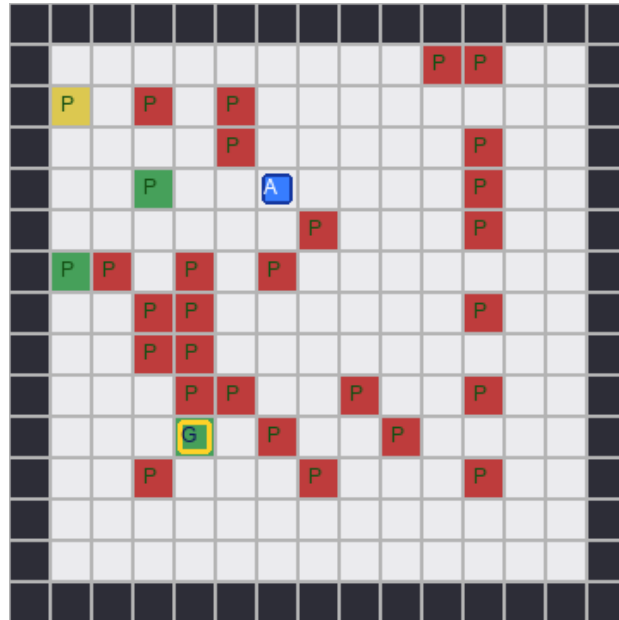
5.6. Ajan Animasyonları ve Görsel Doğrulama

RL ajan davranışları ``evaluation/record_sunum_gif_set.py`` ve ``visualize_agent.py`` ile GIF formatında kaydedilmiştir. PPO ajanı (``output/ppo_agent_explained.gif``) hedefe 12 adımda ulaşırken canlı metrik paneli ödül bileşenlerini gösterir. DQN ajanı (``output/dqn_agent_explained.gif``) sık duvar çarpışması ve timeout davranışı sergiler. Greedy Manhattan baseline (``greedy_shortest_path_baseline_explained.gif``) oracle'a yakın 8.6 adımlık optimal rotayı gösterir. Random baseline ise düzensiz arama paterni ile %47,5 başarı oranına ulaşır.



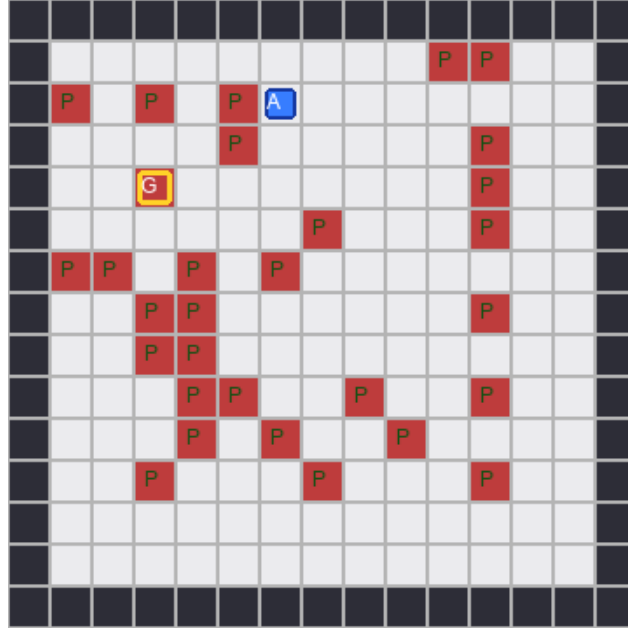
Şekil 45 PPO Eğitim Süreci Başlangıç Davranışı

Şekil 45'te PPO ajanının eğitim sürecinin erken aşamalarındaki davranışları gösterilmektedir. Ajan henüz optimal politikayı öğrenmemiştir.



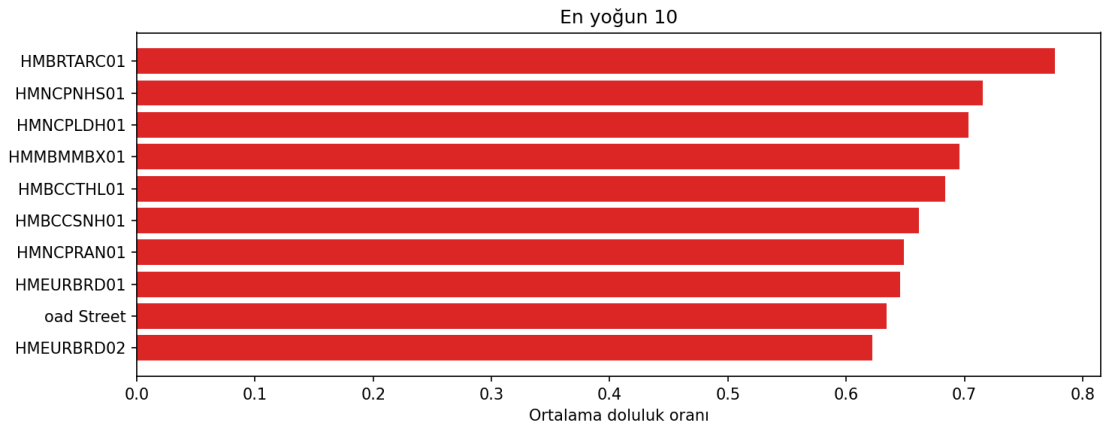
Şekil 46 PPO Eğitim Süreci Orta Aşama

Şekil 46'da PPO ajanının eğitim ilerledikçe daha tutarlı hareketler sergilediği görülmektedir.

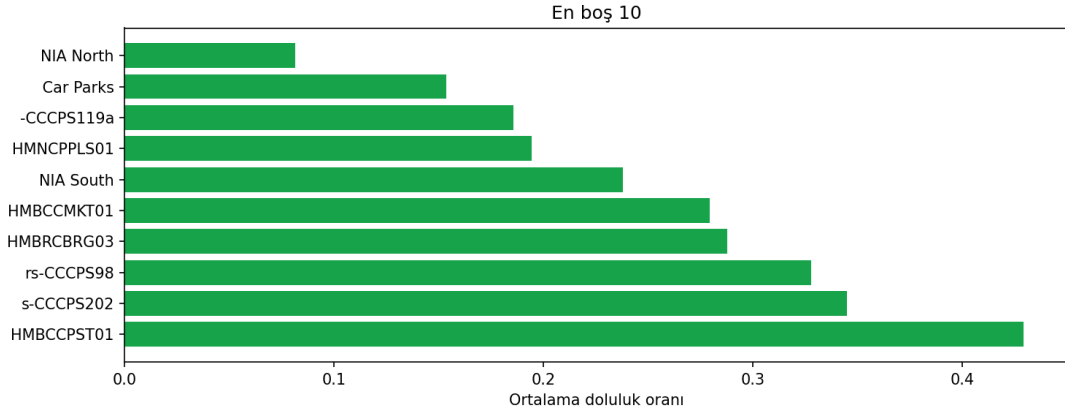


Şekil 47 PPO Eğitim Süreci Sonunda Kararlı Politika

Şekil 47’de PPO ajanının eğitim sonunda oluşturduğu kararlı politika görülmektedir. Ajan hedefe daha kısa ve verimli rotalarla ulaşmaktadır.



Şekil 48 En Yoğun 10 Otopark Lotu Analizi



Şekil 49 En Boş 10 Otopark Lotu Analizi

5.7. Deney Tekrarlanabilirliği

Tüm deneyler RANDOM_SEED=42 ile yürütülmüştür. Veri bölme zaman bazlıdır; gelecek bilgisi eğitim kümesine sızmasını önler. Metrikler `evaluate.py` (100 bölüm) ve `evaluate\_agents.py` (40 bölüm) scriptleri ile bağımsız olarak üretilmiştir. Ham sonuç dosyaları: `evaluation/reports/metrics.json`, `output/rl\_episode\_logs.csv`, `logs/monitor/ppo/mon\_0.monitor.csv`.

6. SONUÇ

Akıllı Park projesi, Parking Birmingham gerçek verisi üzerinde hibrit derin öğrenme ve pekiştirmeli öğrenme mimarisini başarıyla uygulamıştır. Proje kapsamında geliştirilen LSTM modeli, kısa vadeli otopark doluluk tahmini üretmiş; PPO tabanlı ajan ise elde edilen tahminleri karar sürecine dahil ederek grid ortamında sürücüyü düşük yoğunluklu hedef otoparka yönlendirme görevini başarıyla yerine getirmiştir. Gerçekleştirilen deneylerde PPO algoritması yaklaşık %92,5–93 başarı oranına ulaşırken, DQN algoritmasına göre daha kararlı ve etkili sonuçlar üretmiştir. Ayrıca sistem, random baseline yaklaşımına kıyasla anlamlı düzeyde trafik azaltma potansiyeli göstermiştir.

Proje sürecinde veri hazırlama, keşifsel veri analizi, zaman serisi tahmini, pekiştirmeli öğrenme ve kullanıcı arayüzü geliştirme aşamaları bütünsel bir yapı içerisinde gerçekleştirilmiştir. Geliştirilen açıklanabilir ödül mekanizması sayesinde ajanın kararları yorumlanabilir hale getirilmiş, Streamlit tabanlı karar destek paneli ile model çıktıları ve ajan davranışları görsel olarak sunulmuştur. Böylece sistem yalnızca yüksek performanslı sonuçlar üretmekle kalmamış, aynı zamanda karar verme sürecini kullanıcıya şeffaf biçimde aktarabilmiştir.

Hibrit yaklaşımın en önemli avantajı, yalnızca mevcut doluluk durumuna göre değil, gelecekte oluşabilecek yoğunluk değişimlerini de dikkate alarak yönlendirme yapabilmesidir. LSTM modeli tarafından üretilen tahmin sinyallerinin RL ajanının gözlem uzayına dahil edilmesi, sistemin proaktif karar verme yeteneğini artırmıştır. Bu yönüyle proje, klasik otopark yönlendirme sistemlerine göre daha akıllı ve öngörücü bir yaklaşım sunmaktadır.

Gerçek hayata uygulanabilirlik açısından değerlendirildiğinde, proje kapsamında FastAPI tabanlı servis katmanı ve React tabanlı kullanıcı arayüzü geliştirilmiştir. Bu mimari sayesinde belediyeler, üniversite kampüsleri, hastaneler veya büyük alışveriş merkezleri gibi yoğun araç trafiğine sahip alanlarda canlı sensör verileri ile entegre çalışabilecek ölçeklenebilir bir altyapı oluşturulmuştur. Geliştirilen sistem, akıllı şehir uygulamalarında kullanılabilecek karar destek mekanizmalarına yönelik başarılı bir prototip ortaya koymaktadır.

6.1. Gelecek Çalışmalar (Future Work)

- Gerçek zamanlı otopark API entegrasyonu (OpenDataSoft, Kaggle güncellemeleri).
- Canlı trafik verisi ile çoklu hedef optimizasyonu.
- Multi-agent RL: birden fazla aracın aynı grid ortamında rekabet/işbirliği.
- Transformer tabanlı doluluk tahmini ve attention açıklanabilirliği.
- Edge AI: Raspberry Pi / NVIDIA Jetson üzerinde düşük gecikmeli inference.
- IoT sensör ağları ile federated learning tabanlı dağıtık model eğitimi.
- Dynamic goal senaryolarında curriculum learning ile stabilizasyon.
- SUMO veya CARLA ile gerçekçi trafik simülasyonu entegrasyonu.

KAYNAKLAR

- [1] J. Schulman et al., “Proximal Policy Optimization Algorithms,” arXiv preprint arXiv:1707.06347, 2017. Available: <https://arxiv.org/abs/1707.06347> [Eriřim tarihi: 27-Mayıs-2026].
- [2] V. Mnih et al., “Human-level control through deep reinforcement learning,” Nature, vol. 518, pp. 529–533, 2015. Available: <https://www.nature.com/articles/nature14236> [Eriřim tarihi: 27-Mayıs-2026].
- [3] S. Hochreiter and J. Schmidhuber, “Long Short-Term Memory,” Neural Computation, vol. 9, no. 8, pp. 1735–1780, 1997. Available: <https://direct.mit.edu/neco/article/9/8/1735/6109/Long-Short-Term-Memory> [Eriřim tarihi: 27-Mayıs-2026].
- [4] A. Vaswani et al., “Attention Is All You Need,” Advances in Neural Information Processing Systems (NeurIPS), 2017. Available: <https://arxiv.org/abs/1706.03762> [Eriřim tarihi: 27-Mayıs-2026].
- [5] Parking Birmingham Dataset, Kaggle / Open Birmingham Data. Available: <https://www.kaggle.com/datasets/mhmdkardosha/parking-birmingham> [Eriřim tarihi: 27-Mayıs-2026].
- [6] Stable-Baselines3 Contributors, “Stable-Baselines3 Documentation.” Available: <https://stable-baselines3.readthedocs.io/> [Eriřim tarihi: 27-Mayıs-2026].
- [7] F. Pedregosa et al., “Scikit-learn: Machine Learning in Python,” Journal of Machine Learning Research, vol. 12, pp. 2825–2830, 2011. Available: <https://jmlr.org/papers/v12/pedregosa11a.html> [Eriřim tarihi: 27-Mayıs-2026].

[8] T. Chen and C. Guestrin, “XGBoost: A Scalable Tree Boosting System,” Proceedings of the 22nd ACM SIGKDD International Conference on Knowledge Discovery and Data Mining, 2016. Available:

<https://dl.acm.org/doi/10.1145/2939672.2939785>

[Eriřim tarihi: 27-Mayıs-2026].

[9] Gymnasium Contributors, “Gymnasium Documentation,” Farama Foundation.

Available: <https://gymnasium.farama.org/>

[Eriřim tarihi: 27-Mayıs-2026].

[10] M. B. Menne et al., “Smart Parking Systems: Review and Future Directions,” IEEE Access, vol. 9, pp. 78956–78973, 2021.

[11] PyTorch Team, “PyTorch Documentation.” Available:

<https://pytorch.org/docs/>

[Eriřim tarihi: 27-Mayıs-2026].

[12] Streamlit Inc., “Streamlit Documentation.” Available:

<https://docs.streamlit.io/>

[Eriřim tarihi: 27-Mayıs-2026].

[13] University of Bath, “Referencing guide: IEEE.” Available:

<https://library.bath.ac.uk/referencing/ieee>

[Eriřim tarihi: 27-Mayıs-2026].

PROJE GİTHUB REPO: https://github.com/Yapay-Sinir-Aglari-Proje/Akilli_Park